

Lecture: 05

Size of vector clock, Matrix clocks, Virtual time and Physical clock synchronization



Rajiv Misra

Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture we have discussed the models of distributed computation, and presented the idea of causality & logical time that was proposed by Lamport in 1978 in an attempt to order events in distributed systems.
- We have discussed two systems of logical clocks, namely, scalar and vector clocks to capture causality between events of a distributed computation.

Preface

Content of this Lecture:

In this lecture we will discuss about:

- Size of vector clocks
- Matrix clocks
- Virtual time and
- Physical clock synchronization

Size of vector clocks

- An important question to ask is whether vector clocks of size n are necessary in a computation consisting of n processes. To answer this, we examine the usage of vector clocks.
- A vector clock provides the latest known local time at each other process. If this information in the clock is to be used to explicitly track the progress at every other process, then a vector clock of size n is necessary.
- A popular use of vector clocks is to determine the causality between a pair of events. Given any events e and f , the test for $e \prec f$ if and only if $T(e) < T(f)$, which requires a comparison of the vector clocks of e and f .
- Although it appears that the clock of size n is necessary, that is not quite accurate. It can be shown that a size equal to the dimension of the partial order $(E, \prec)$ is necessary, where the upper bound on this dimension is n .

Size is equal to dimension of Partial Order

Definitions:

- To understand this result on the size of clocks for determining causality between a pair of events, we first introduce some definitions:
- A **linear extension** of a partial order $(E, <)$ is a linear ordering of E that is consistent with the partial order, i.e. if two events are ordered in the partial order, they are also ordered in the linear order.
- A linear extension can be viewed as projecting all the events from the different processes on a single time axis.
- The **dimension** of a partial order is the minimum number of linear extensions, whose intersection gives exactly the partial order.

Contd...

- However, the linear order will necessarily introduce ordering between each pair of events, and some of these orderings are not in the partial order. Also observe that different linear extensions are possible in general.
- Let $\mathcal{P}$ denote the **set of tuples** in the partial order defined by the causality relation; so there is a **tuple (e, f)** in $\mathcal{P}$ for each pair of **events e and f** such that **$e < f$** . Let $\mathcal{L}_1, \mathcal{L}_2, \dots$ denote the **sets of tuples in different linear extensions** of this partial order.
- The set $\mathcal{P}$ is contained in the set obtained by taking the intersection of any such collection of linear extensions $\mathcal{L}_1, \mathcal{L}_2, \dots$. This is because each $\mathcal{L}_i$ must contain all the tuples, i.e., causality dependencies, that are in $\mathcal{P}$.

1. Client-Server Interaction

- Consider a client–server interaction between a pair of processes. Queries to the server and responses to the client occur in strict alternating sequences.
- Although $n=2$, all the events are strictly ordered, and there is only one linear order of all the events that is consistent with the “partial” order.
- Hence the dimension of this “partial order” is 1. A scalar clock such as one implemented by Lamport’s scalar clock rules is adequate to determine $e < f$ for any events e and f in this execution.

2. Concurrent Send-Receive

- Now consider an execution on processes P_1 and P_2 such that each sends a message to the other before receiving the other's message.
- The two send events are concurrent, as are the two receive events. To determine the causality between the send events or between the receive events, it is not sufficient to use a single integer; a vector clock of size $n = 2$ is necessary.
- This execution exhibits the graphical property called a **crown**, wherein there are some messages $m_0, \dots, m_{n-1}$ such that **$\text{Send}(m_i) < \text{Receive}(m_{i+1 \bmod (n-1)})$** for all i from 0 to $n-1$. A crown of n messages has dimension n .

3. Complex Execution

- For a complex execution, it is not straightforward to determine the dimension of the partial order. Figure 5.1 shows an execution involving four processes. However, the dimension of this partial order is two.
- To see this informally, consider the **longest chain** $\langle a, b, d, g, h, i, j \rangle$. There are events outside this chain that can yield multiple linear extensions. Hence, the dimension is more than 1.
- The right side of Figure 5.1 shows the earliest possible and the latest possible occurrences of the events not in this chain, with respect to the events in this chain.

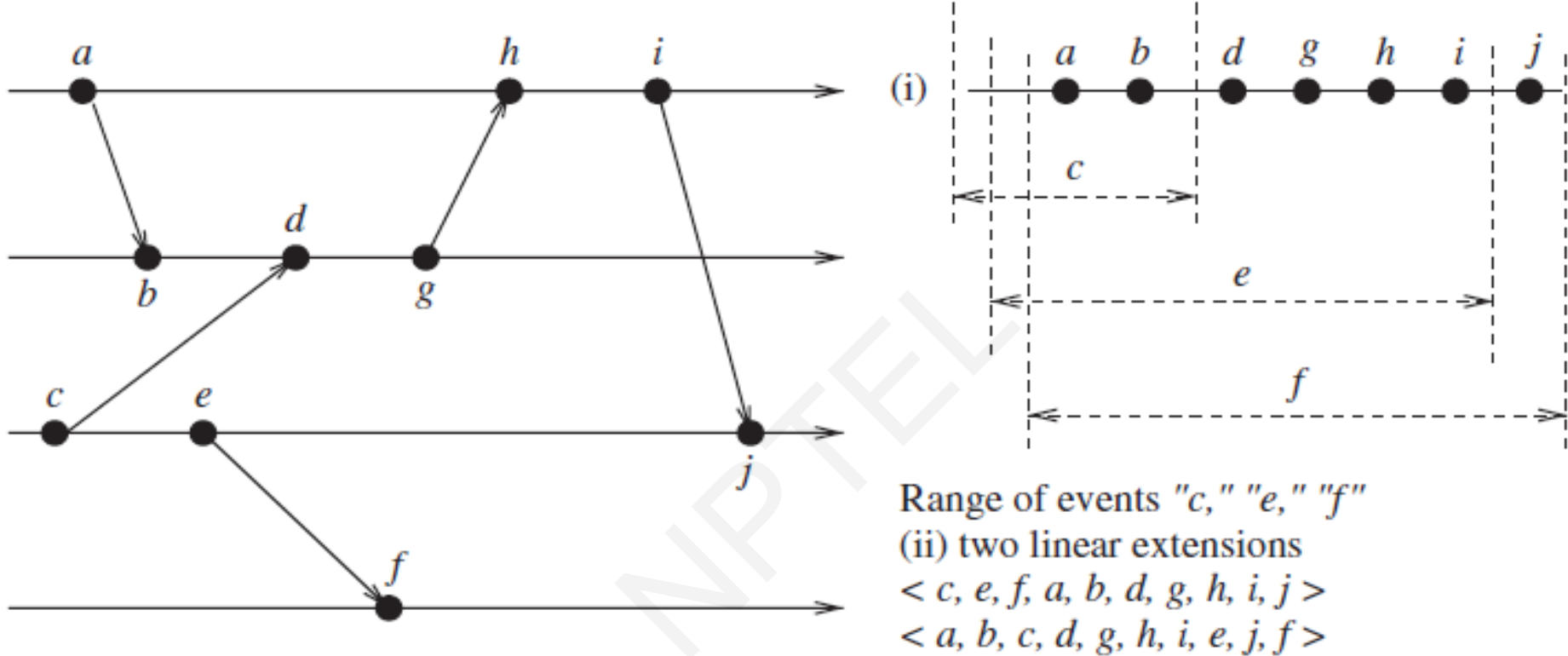


Figure 5.1 Example illustrating dimension of an execution $(E, <)$.

For $n = 4$ processes, the dimension is 2.

3. Complex Execution

Let $\mathcal{L}_1$ be $\langle c, e, f, a, b, d, g, h, i, j \rangle$ which contains the following tuples that are not in $\mathcal{P}$:

$(c, a), (c, b), (c, d), (c, g), (c, h), (c, i), (c, j)$
 $(e, a), (e, b), (e, d), (e, g), (e, h), (e, i), (e, j)$
 $(f, a), (f, b), (f, d), (f, g), (f, h), (f, i), (f, j)$

Let $\mathcal{L}_2$ be $\langle a, b, c, d, g, h, i, e, j, f \rangle$, which contains the following tuples not in $\mathcal{P}$:

$(a, c), (b, c), (c, d), (c, g), (c, h), (c, i), (c, j)$
 $(a, e), (b, e), (d, e), (g, e), (h, e), (i, e), (e, j)$
 $(a, f), (b, f), (d, f), (g, f), (h, f), (i, f), (j, f)$

- Further, observe that $(\mathcal{L}_1 \setminus \mathcal{P}) \cap \mathcal{L}_2 = \emptyset$ and $(\mathcal{L}_2 \setminus \mathcal{P}) \cap \mathcal{L}_1 = \emptyset$.
Hence, $\mathcal{L}_1 \cap \mathcal{L}_2 = \mathcal{P}$ and the dimension of the execution is 2 as these two linear extensions are enough to generate $\mathcal{P}$.

Matrix Time

- In a system of matrix clocks, the time is represented by a set of $n \times n$ matrices of non-negative integers.
- A process p_i maintains a matrix $mt_i[1..n, 1..n]$ where,
 - $mt_i[i, i]$ denotes the local logical clock of p_i and tracks the progress of the computation at process p_i .
 - $mt_i[i, j]$ denotes the latest knowledge that process p_i has about the local logical clock, $mt_j[j, j]$, of process p_j .
 - $mt_i[j, k]$ represents the knowledge that process p_i has about the latest knowledge that p_j has about the local logical clock, $mt_k[k, k]$, of p_k .
- The entire matrix mt_i denotes p_i 's local view of the global logical time.

Matrix Time

Process p_i uses the following rules $R1$ and $R2$ to update its clock:

$R1$: Before executing an event, process p_i updates its local logical time as follows:

$$mt_i[i, i] := mt_i[i, i] + d \quad (d > 0)$$

$R2$: Each message m is piggybacked with matrix time mt . When p_i receives such a message (m, mt) from a process p_j , p_i executes the following sequence of actions:

- Update its global logical time as follows:

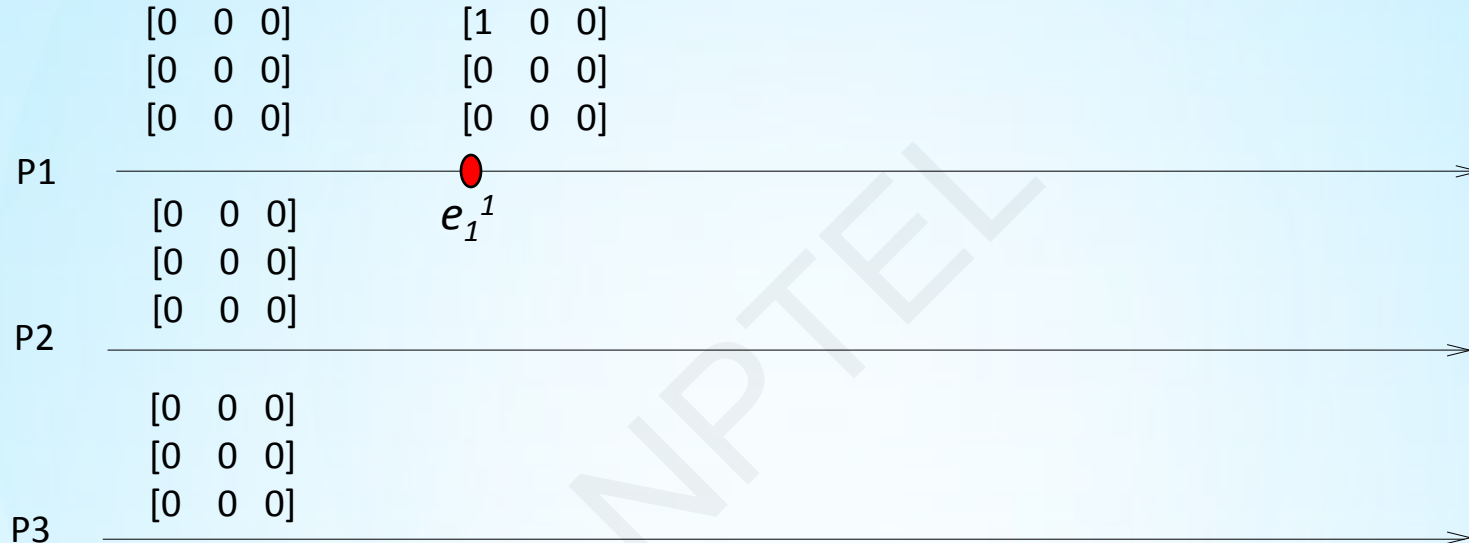
$$(a) \ 1 \leq k \leq n : mt_i[i, k] := \max(mt_i[i, k], mt[j, k])$$

(That is, update its row $mt_i[i, *]$ with the p_j 's row in the received timestamp, mt)

$$(b) \ 1 \leq k, l \leq n : mt_i[k, l] := \max(mt_i[k, l], mt[k, l])$$

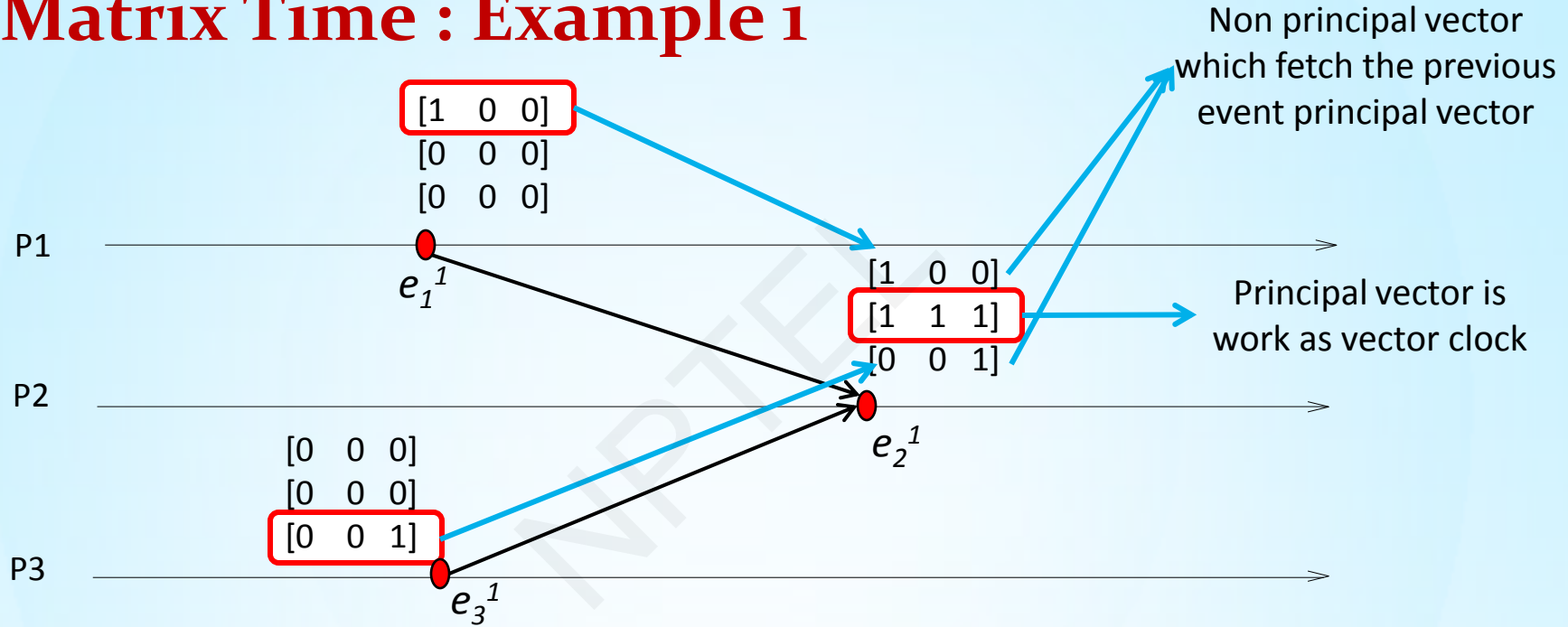
- Execute $R1$
- Deliver message m

Matrix Time : Example 1



Initial Form of Matrix Clock

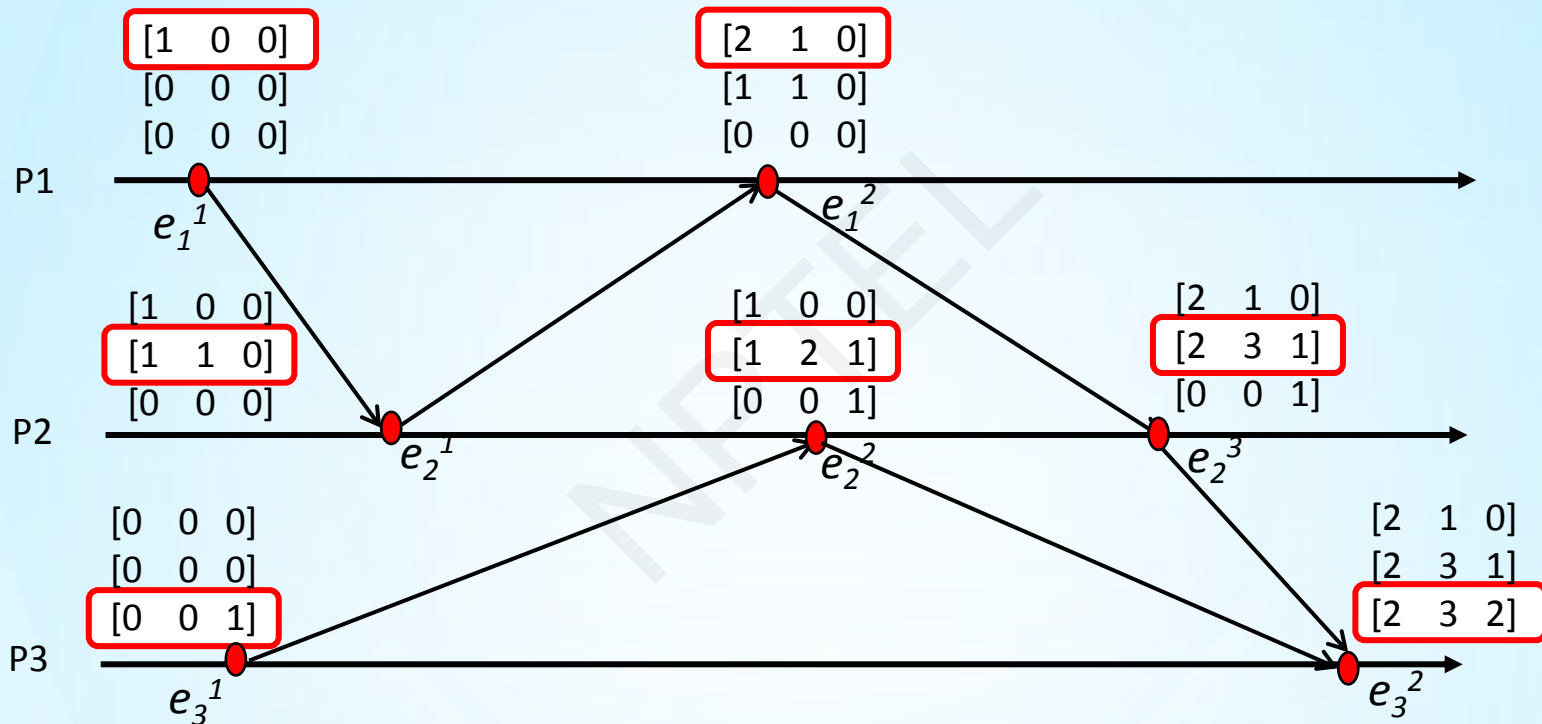
Matrix Time : Example 1



Working of principal and non-principal vector in matrix clock

Matrix Time : Example 2

 → Principal vector



Working of principal and non-principal vector in matrix clock

Basic Properties

- Vector $mt_i[j, .]$ contains all the properties of vector clocks. In addition, matrix clocks have the following property:
- **$\min_k (mt_i[k, l]) \geq t \Rightarrow$ process p_i** knows that every other process p_k knows that p_i 's local time has progressed till t .
- If this is true, it is clear that process p_i knows that all other processes know that p_i will never send information with a local time $\leq t$.
- In many applications, this implies that processes will no longer require from p_i certain information and can use this fact to discard obsolete information.
- If d is always 1 in the rule $R1$, then $mt_i[k, l]$ denotes the number of events occurred at p_i and known by p_k as far as p_i 's knowledge is concerned.

Virtual Time

- Virtual time system is a paradigm for organizing and synchronizing distributed systems.
- This section provides description of virtual time and its implementation using the **Time Warp mechanism**.
- The implementation of virtual time using Time Warp mechanism works on the basis of an optimistic assumption.
- Time Warp relies on the **general lookahead-rollback mechanism** where each process executes without regard to other processes having synchronization conflicts.

Virtual Time

- If a conflict is discovered, the offending processes are rolled back to the time just before the conflict and executed forward along the revised path.
- Detection of conflicts and rollbacks are transparent to users.
- The implementation of Virtual Time using Time Warp mechanism makes the following optimistic assumption: synchronization conflicts and thus rollbacks generally occurs rarely.

Virtual Time

- “Virtual time is a global, one dimensional, temporal coordinate system on a distributed computation to measure the computational progress and to define synchronization.”
- A virtual time system is a distributed system executing in coordination with an imaginary virtual clock that uses virtual time.
- Virtual times are real values that are totally ordered by the less than relation, “ $<$ ”.
- Virtual time is implemented a collection of several loosely synchronized local virtual clocks.
- These local virtual clocks move forward to higher virtual times; however, occasionally they move backwards.

Virtual Time: *Definition*

- Processes run concurrently and communicate with each other by exchanging messages.
- Every message is characterized by four values:
 - a) *Name of the sender*
 - b) *Virtual send time*
 - c) *Name of the receiver*
 - d) *Virtual receive time*
- Virtual send time is the virtual time at the sender when the message is sent, whereas virtual receive time specifies the virtual time when the message must be received (and processed) by the receiver.

Virtual Time: *Definition*

- A problem arises when a message arrives at process late, that is, the virtual receive time of the message is less than the local virtual time at the receiver process when the message arrives.
- Virtual time systems are subject to **two semantic rules** similar to Lamport's clock conditions:
 - Rule 1:** Virtual send time of each message $<$ Virtual receive time of that message.
 - Rule 2:** Virtual time of each event in a process $<$ Virtual time of next event in that process.
- The above two rules imply that a process sends all messages in increasing order of virtual send time and a process receives (and processes) all messages in the increasing order of virtual receive time.

Virtual Time: *Definition*

- It is important an event that causes another should be completely executed before the caused event can be processed.
- The constraint in the implementation of virtual time can be stated as follows:

“If an event A causes event B, then the execution of A and B must be scheduled in real time so that A is completed before B starts”.

Virtual Time: *Definition*

- If event A has an earlier virtual time than event B, we need execute A before B provided there is no causal chain from A to B.
- Better performance can be achieved by scheduling A concurrently with B or scheduling A after B.
- If A and B have exactly the same virtual time coordinate, then there is no restriction on the order of their scheduling.
- If A and B are distinct events, they will have different virtual space coordinates (since they occur at different processes) and neither will be a cause for the other.
- To sum it up, events with virtual time $< 't'$ complete before the starting of events at time $'t'$ and events with virtual time $> 't'$ will start only after events at time $'t'$ are complete.

Characteristics of Virtual Time

- Virtual time systems are not all isomorphic; it may be either discrete or continuous.
- Virtual time may be only partially ordered.
- Virtual time may be related to real time or may be independent of it.
- Virtual time systems may be visible to programmers and manipulated explicitly as values, or hidden and manipulated implicitly according to some system-defined discipline
- Virtual times associated with events may be explicitly calculated by user programs or they may be assigned by fixed rules.

Comparison with Lamport's Logical Clocks

- In Lamport's logical clock, an artificial clock is created one for each process with unique labels from a totally ordered set in a manner consistent with partial order.
- In virtual time, the reverse of the above is done by assuming that every event is labeled with a clock value from a totally ordered virtual time scale satisfying Lamport's clock conditions.
- Thus the *Time Warp mechanism* is an inverse of Lamport's scheme.
- In Lamport's scheme, all clocks are conservatively maintained so that they never violate causality.
- A process advances its clock as soon as it learns of new causal dependency. In the virtual time, clocks are optimistically advanced and corrective actions are taken whenever a violation is detected.
- Lamport's initial idea brought about the concept of virtual time but the model failed to preserve causal independence.

Time Warp Mechanism

- In the implementation of virtual time using Time Warp mechanism, virtual receive time of message is considered as its timestamp.
- The necessary and sufficient conditions for the correct implementation of virtual time are that each process must handle incoming messages in *timestamp* order.
- This is highly undesirable and restrictive because process speeds and message delays are likely to be highly variable.
- It is natural for some processes to get ahead in virtual time of other processes.

Time Warp Mechanism

- It is impossible for a process on the basis of local information alone to block and wait for the message with the next timestamp.
- It is always possible that a message with earlier timestamp arrives later.
- So, when a process executes a message, it is very difficult for it determine whether a message with an earlier timestamp will arrive later.
- This is the central problem in virtual time that is solved by the Time Warp mechanism.
- The Time warp mechanism assumes that message communication is reliable, and messages may not be delivered in FIFO order.

Time Warp Mechanism

Time Warp mechanism consists of two major parts:

(i) local control mechanism and (ii) global control mechanism

(i) The **local control mechanism** ensures that events are executed and messages are processed in the correct order.

(ii) The **global control mechanism** takes care of global issues such as global progress, termination detection, I/O error handling, flow control, etc.

Physical Clock Synchronization: NTP

- In centralized systems, there is only single clock. A process gets the time by simply issuing a system call to the kernel.
- In distributed systems, there is no global clock or common memory. Each processor has its own internal clock and its own notion of time.
- These clocks can easily drift seconds per day, accumulating significant errors over time.
- Also, because different clocks tick at different rates, they may not remain always synchronized although they might be synchronized when they start.
- This clearly poses serious problems to applications that depend on a synchronized notion of time.

Physical Clock Synchronization: NTP

For most applications and algorithms that run in a distributed system, we need to know time in one or more of the following contexts:

- **The time of the day** at which an event happened on a specific machine in the network.
- **The time interval** between two events that happened on different machines in the network.
- **The relative ordering of events** that happened on different machines in the network.
- Unless the clocks in each machine have a common notion of time, time-based queries cannot be answered.
- **Clock synchronization** has a significant effect on many problems like secure systems, fault diagnosis and recovery, scheduled operations, database systems, and real-world clock values.

Physical Clock Synchronization: NTP

- Clock synchronization is the process of ensuring that physically distributed processors have a common notion of time.
- Due to different clocks rates, the clocks at various sites may diverge with time and periodically a clock synchronization must be performed to correct this clock skew in distributed systems.
- Clocks are synchronized to an accurate real-time standard like **UTC (Universal Coordinated Time)**.
- Clocks that must not only be synchronized with each other but also have to adhere to physical time are termed ***physical clocks***.

Clock Inaccuracies

- Physical clocks are synchronized to an accurate real-time standard like UTC (Universal Coordinated Time).
- However, due to the clock inaccuracy discussed above, **a timer (clock)** is said to be working within its specification if (where **constant ρ** is the **maximum skew rate** specified by the manufacturer.)

$$1 - \rho \leq \frac{dc}{dt} \leq 1 + \rho \quad (1)$$

- Figure 5.3 illustrates the behavior of **fast, slow, and perfect clocks** with respect to UTC.

Clock Inaccuracies

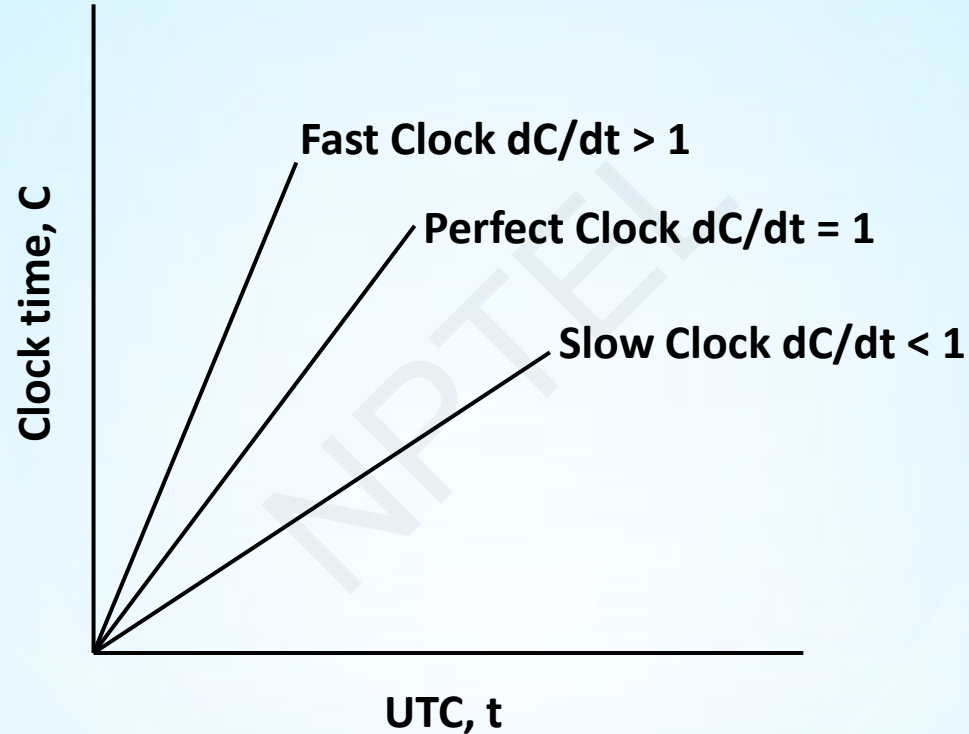


Figure 5.3: The behavior of fast, slow, and perfect clocks with respect to UTC.

Offset delay estimation method

- The **Network Time Protocol (NTP)** which is widely used for clock synchronization on the Internet uses the **Offset Delay Estimation** method.
- The design of NTP involves a **hierarchical tree** of time servers:
 - (i) The primary server at the root synchronizes with the UTC.
 - (ii) The next level contains secondary servers, which act as a backup to the primary server.
 - (iii) At the lowest level is the synchronization subnet which has the clients.

Clock offset and delay estimation:

- In practice, a source node cannot accurately estimate the local time on the target node due to varying message or network delays between the nodes.
- This protocol employs a common practice of performing several trials and chooses the trial with the minimum delay.
- Figure 5.4 shows how **NTP timestamps** are numbered and exchanged between *peers A and B*.
- Let T_1, T_2, T_3, T_4 be the values of the *four most recent timestamps* as shown. Assume **clocks A and B are stable** and **running at the same speed**.

Clock offset and delay estimation:

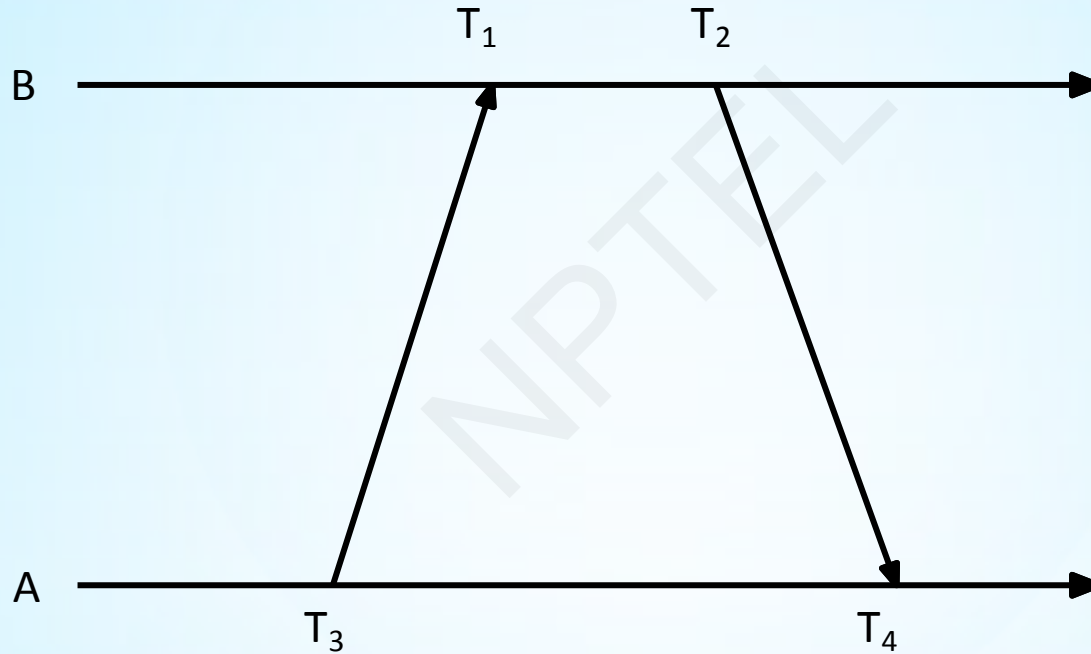


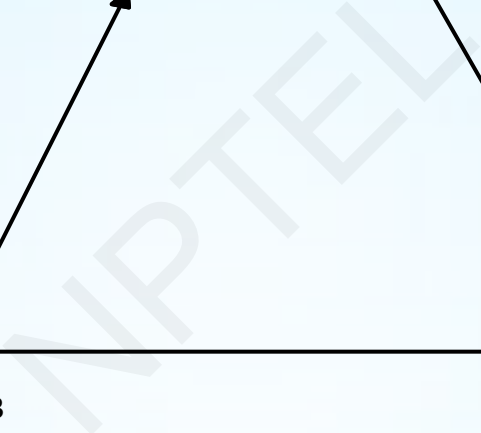
Figure 5.4: Offset and delay estimation

Clock offset and delay estimation:

- Let $a = T_1 - T_3$ and $b = T_2 - T_4$.
- If the network delay difference from A to B and from B to A , called **differential delay**, is small, the clock offset θ and roundtrip delay δ of B relative to A at time T_4 are approximately given by the following:

$$\theta = \frac{a + b}{2}, \quad \delta = a - b \quad (2)$$

- Each NTP message includes the latest three timestamps T_1 , T_2 and T_3 , while T_4 is determined upon arrival.
- Thus, both peers A and B can independently calculate delay and offset using a single bidirectional message stream as shown in Figure 5.5.



The Network Time Protocol: Synchronization protocol

- A pair of servers in symmetric mode exchange pairs of timing messages.
- A store of data is then built up about the relationship between the two servers (pairs of offset and delay).
- Specifically, assume that each peer maintains **pairs (O_i, D_i)** , where
- **O_i** - measure of offset (**θ**)
- **D_i** - transmission delay of two messages (**δ**).
- The offset corresponding to the minimum delay is chosen.
- Specifically, the delay and offset are calculated as follows. Assume that **message m** takes **time t** to transfer and **m'** takes **t'** to transfer.

Contd...

The offset between A's clock and B's clock is O .

If **A's local clock time is $A(t)$** and **B's local clock time is $B(t)$** , we have

$$A(t) = B(t) + O \quad (3)$$

Then,

$$T_{i-2} = T_{i-3} + t + O \quad (4)$$

$$T_i = T_{i-1} - O + t' \quad (5)$$

Assuming **$t = t'$** , the **offset O_i** can be estimated as:

$$O_i = (T_{i-2} - T_{i-3} + T_{i-1} - T_i) / 2 \quad (6)$$

The **round-trip delay D_i** is estimated as:

$$D_i = (T_i - T_{i-3}) - (T_{i-1} - T_{i-2}) \quad (7)$$

The eight most recent pairs of (O_i, D_i) are retained.

The value of O_i that corresponds to minimum D_i is chosen to estimate O

Q: Whether Physical Clock or Logical Clock is used to capture the 'Causality' (In Distributed System)?

- In day-to-day life, the global time(physical time) to deduce **causality relation** is obtained from **loosely synchronized clocks** (i.e., wrist watches, wall clocks).
- However, in distributed computing systems, the rate of occurrence of events is several magnitudes higher and the event execution time is several magnitudes smaller. Consequently, if the **physical clocks** are not precisely synchronized, the causality relation between events may not be accurately captured.
- Therefore, it turns out that in a distributed computation, the causality relation between events produced by a program execution and its fundamental monotonicity property can be accurately captured by **logical clocks**.

Conclusion

- In this lecture we have discussed about the **size of vector clock, matrix clocks and virtual time** to capture causality between events of a distributed computation and
- How the **physical clock synchronization** can be used as a paradigm for organizing and synchronizing distributed systems.
- In upcoming lecture, we will discuss about Global state and snapshot recording algorithms.

Lecture: 06

Global State and Snapshot Recording Algorithms



Rajiv Misra

Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about the models of distributed computation, causality and a general framework of logical clocks (i.e. scalar, vector and matrix clocks) in distributed systems and
- How the virtual time and physical clock synchronization can be used as a paradigm for organizing and synchronizing distributed systems.

Preface

Content of this Lecture:

- In this lecture, we will discuss about the Global states (i.e. consistent, inconsistent), Cuts in the space time diagram, Models of communication and
- Snapshot algorithm *i.e.* Chandy-Lamport algorithm to record the global snapshot.

Global State: Introduction

- Recording the global state of a distributed system on-the-fly is an important paradigm.
- The lack of globally shared memory, global clock and unpredictable message delays in a distributed system make this problem non-trivial.
- This lecture first defines consistent global states and discusses issues to be addressed to compute consistent distributed snapshots.
- Then the algorithm to determine on-the-fly such snapshots is presented.

System Model

- The system consists of a collection of n processes $p_1, p_2, \dots, p_n$ that are connected by channels.
- There are no globally shared memory and physical global clock and processes communicate by passing messages through communication channels.
- C_{ij} denotes the channel from process p_i to process p_j and its state is denoted by SC_{ij} .
- The actions performed by a process are modeled as three types of events: Internal events, the message send event and the message receive event.
- For a message m_{ij} that is sent by process p_i to process p_j , let $send(m_{ij})$ and $rec(m_{ij})$ denote its send and receive events.

System Model

- At any instant, the state of process p_i , denoted by LS_i , is a result of the sequence of all the events executed by p_i till that instant.
- For an event e and a process state LS_i , $e \in LS_i$ iff e belongs to the sequence of events that have taken process p_i to state LS_i .
- For an event e and a process state LS_i , $e \notin LS_i$ iff e does not belong to the sequence of events that have taken process p_i to state LS_i .
- For a channel C_{ij} , the following set of messages can be defined based on the local states of the processes p_i and p_j

Transit: $transit(LS_i, LS_j) = \{m_{ij} \mid send(m_{ij}) \in LS_i \wedge rec(m_{ij}) \notin LS_j\}$

Consistent Global State

- The global state of a distributed system is a collection of the local states of the processes and the channels.
- Notationally, global state GS is defined as,
$$GS = \{U_i LS_i, U_{i,j} SC_{ij}\}$$
- A global state GS is a **consistent global state** iff it satisfies the following two conditions :

C1: $\text{send}(m_{ij}) \in LS_i \Rightarrow m_{ij} \in SC_{ij} \oplus \text{rec}(m_{ij}) \in LS_j$
($\oplus$ is Ex-OR operator)

C2: $\text{send}(m_{ij}) \notin LS_i \Rightarrow m_{ij} \notin SC_{ij} \wedge \text{rec}(m_{ij}) \notin LS_j$

Global State of a Distributed System

- In the distributed execution of Figure 6.2:
- A global state GS_1 consisting of local states $\{LS_1^1, LS_2^3, LS_3^3, LS_4^2\}$ is **inconsistent** because the state of p_2 has recorded the receipt of message m_{12} , however, the state of p_1 has not recorded its send.
- On the contrary, a global state GS_2 consisting of local states $\{LS_1^2, LS_2^4, LS_3^4, LS_4^2\}$ is **consistent**; all the channels are empty except c_{21} that contains message m_{21} .

Global State of a Distributed System

- A global state $GS = \{U_i LS_i^{xi} , U_{j,k} SC_{jk}^{yj,zk} \}$ is transitless iff

$$\text{iff } \forall i, \forall j : 1 \leq i, j \leq n : SC_{jk}^{yj,zk} = \emptyset$$

- Thus, all channels are recorded as empty in a transitless global state. A global state is **strongly consistent** iff it is transitless as well as consistent. Note that in figure 6.2, the global state of local states $\{LS_1^2, LS_2^3, LS_3^4, LS_4^2\}$ is **strongly consistent**.
- Recording the global state of a distributed system is an important paradigm when one is interested in analyzing, monitoring, testing, or verifying properties of distributed applications, systems, and algorithms. Design of efficient methods for recording the global state of a distributed system is an important problem.

Example:

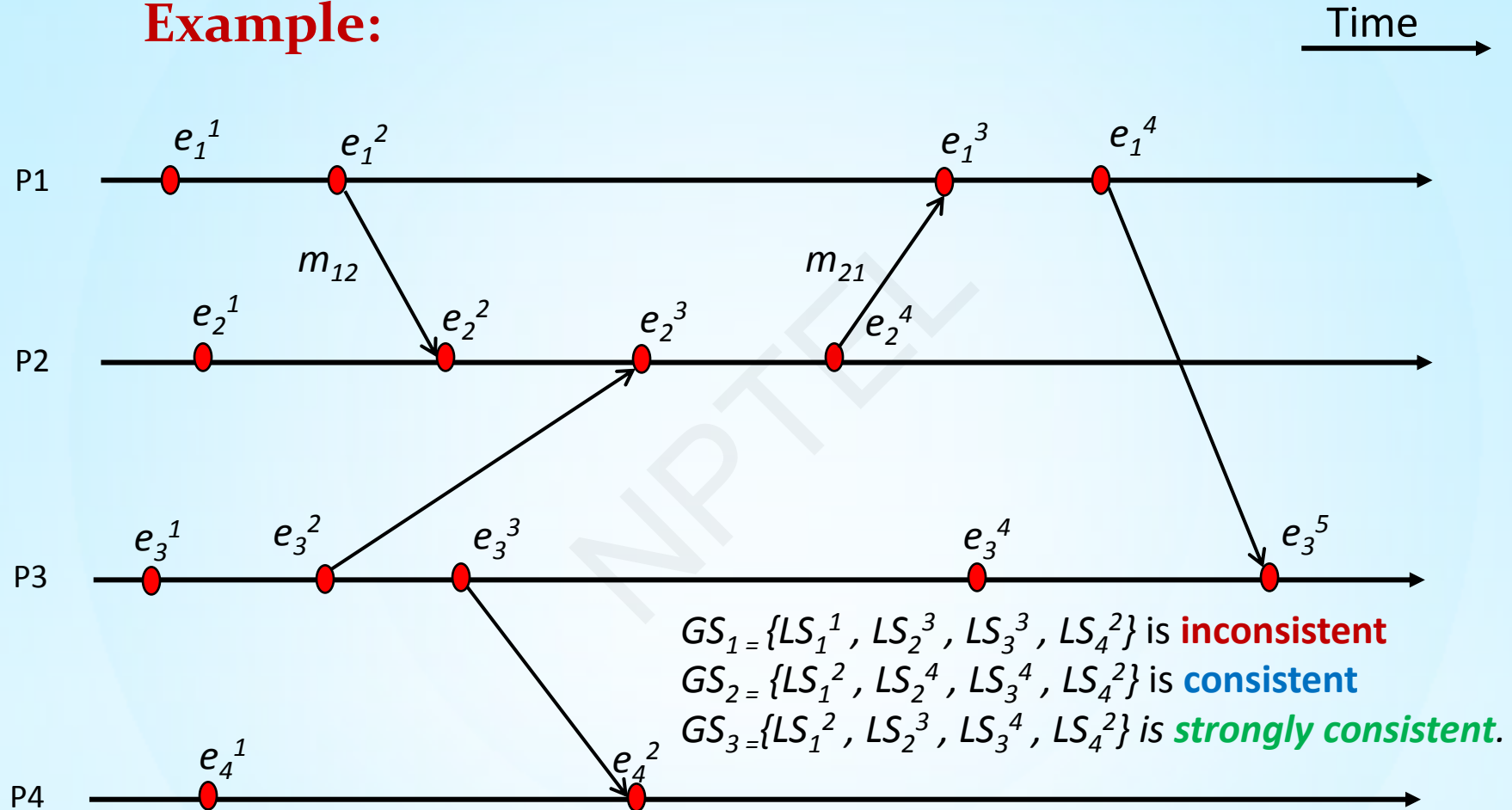


Figure 6.2: The space-time diagram of a distributed execution.

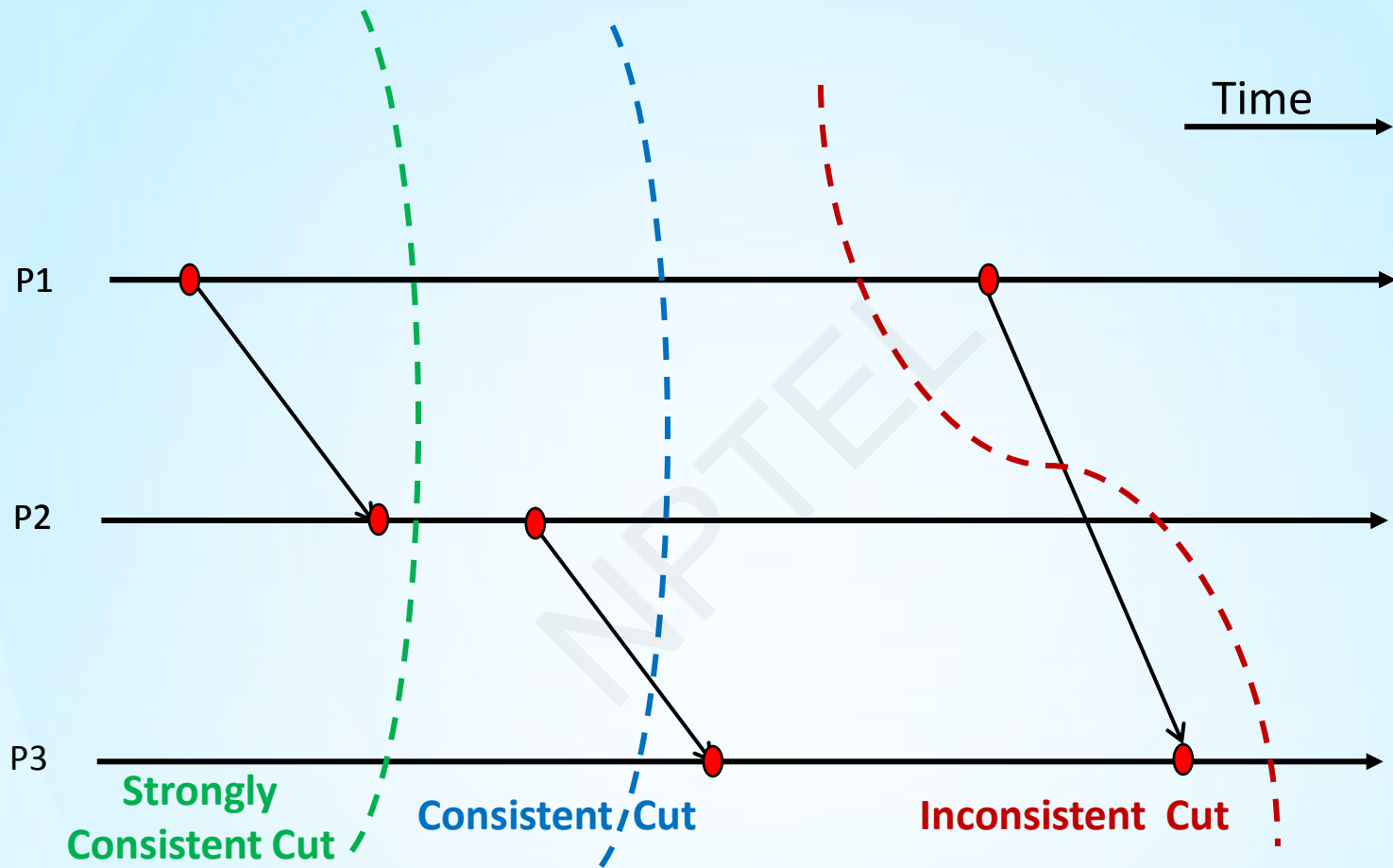


Figure 6.1 Cuts of a distributed computation

Cuts of a distributed computation

- In the space–time diagram of a distributed computation, a **zigzag line** joining one arbitrary point on each process line is termed a **cut in the computation**.
- Such a line slices the space–time diagram, and thus the set of events in the distributed computation, into a **PAST** and a **FUTURE**. The **PAST** contains all the events to the left of the cut and the **FUTURE** contains all the events to the right of the cut. For **a cut C**, let **PAST(C)** and **FUTURE(C)** denote the set of events in the PAST and FUTURE of C, respectively.
- Every cut corresponds to a global state and every global state can be graphically represented as a cut in the computation's space–time diagram.
- Cuts in a space-time diagram provide a powerful graphical aid in representing and reasoning about global states of a computation.

Cuts of a Distributed Computation

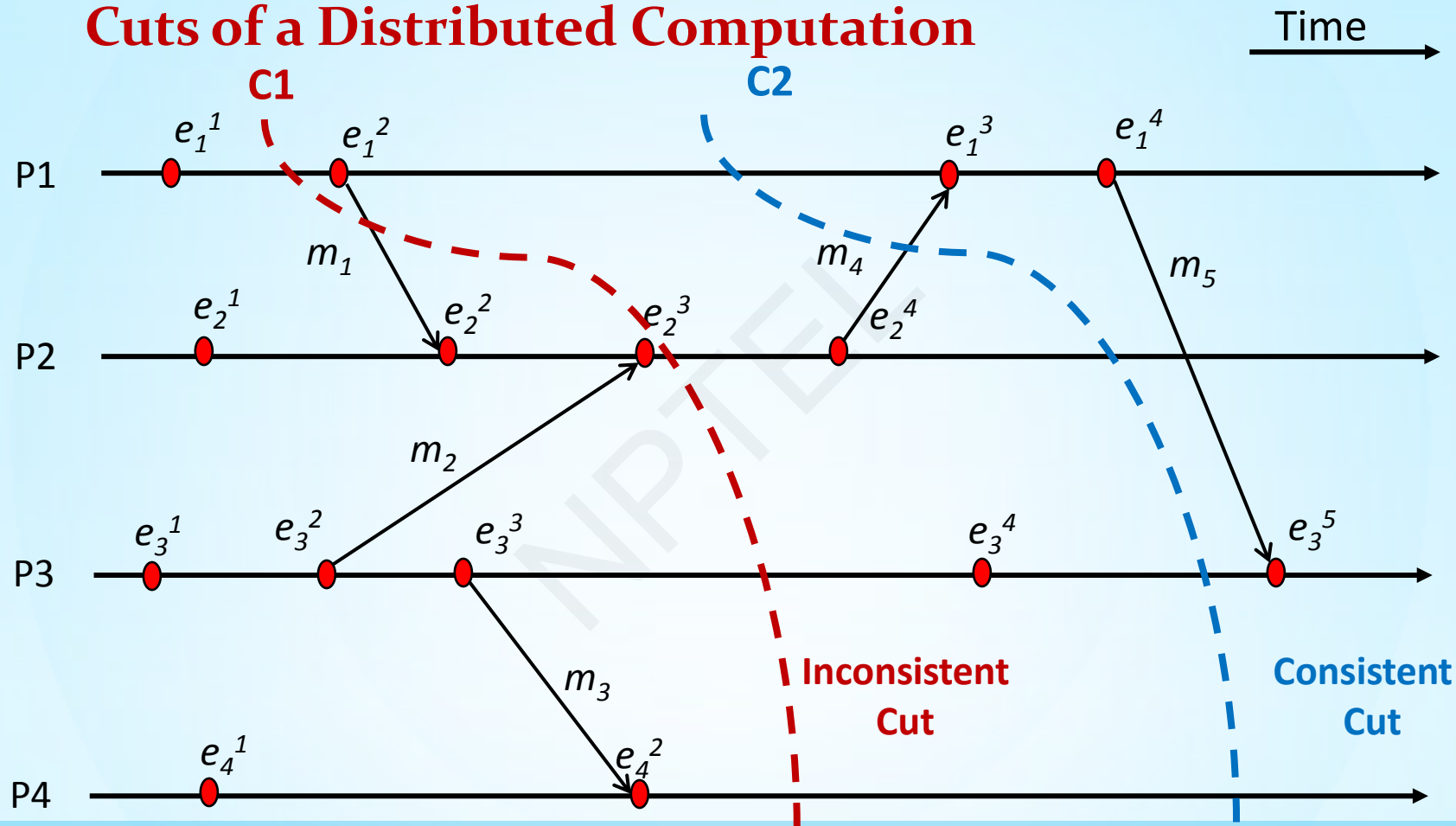


Figure 6.2: Illustration of Cut in a distributed execution.

Explanation of Cut in a distributed execution

- In a consistent cut, every message received in the PAST of the cut was sent in the PAST of that cut. (In Figure 6.2, **cut C_2 is a consistent cut.**)
- All messages that cross the cut from the PAST to the FUTURE are in transit in the corresponding consistent global state.
- A cut is *inconsistent* if a message crosses the cut from the FUTURE to the PAST. (In Figure 6.2, **cut C_1 is an inconsistent cut.**)

Issues in Recording a Global State

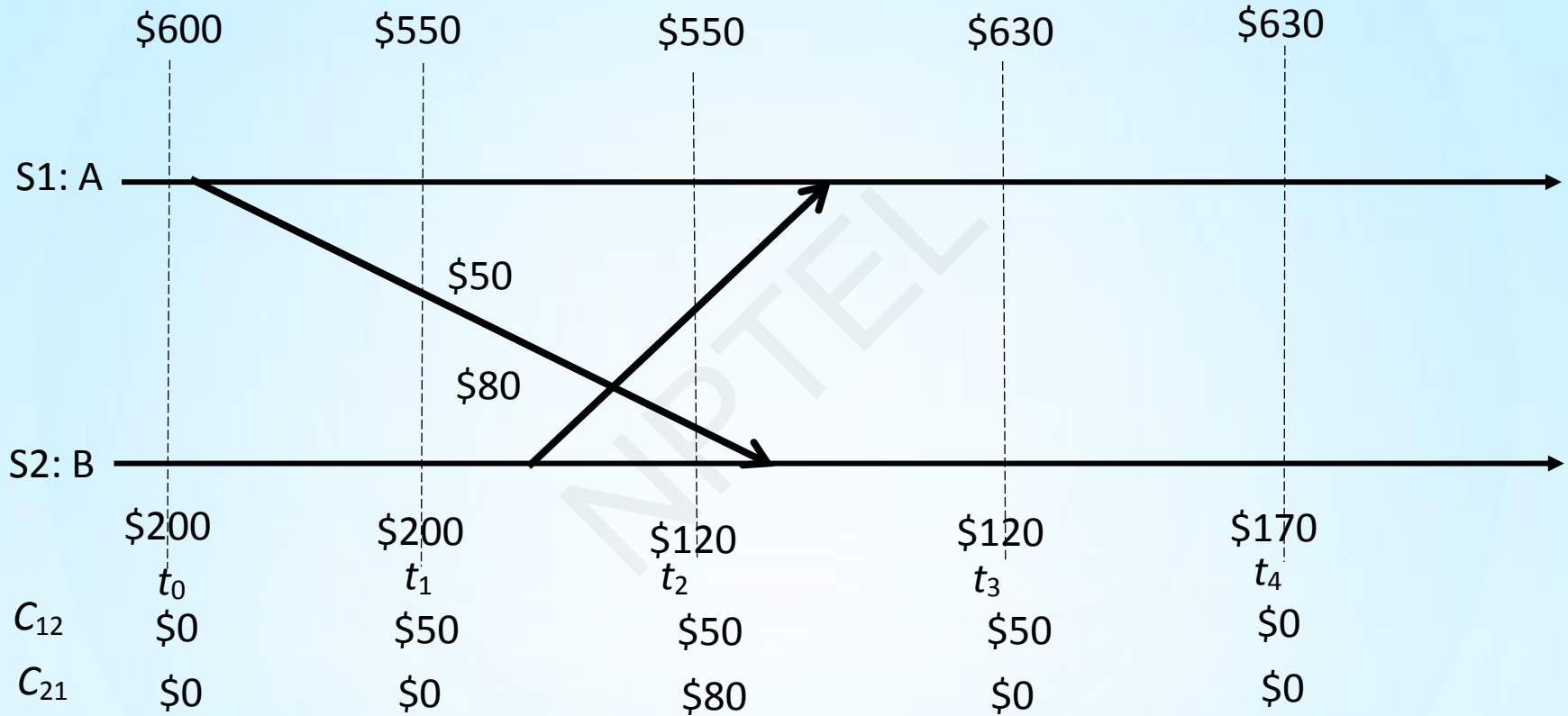
- The following two issues need to be addressed:
 - I1:** How to distinguish between the messages to be recorded in the snapshot from those not to be recorded.
 - -Any message that is sent by a process before recording its snapshot, must be recorded in the global snapshot (from **C1**).
 - -Any message that is sent by a process after recording its snapshot, must not be recorded in the global snapshot (from **C2**).
 - I2:** How to determine the instant when a process takes its snapshot.
 - -A process p_j must record its snapshot before processing a message m_{ij} that was sent by process p_i after recording its snapshot.

Example of Money Transfer

- Let S_1 and S_2 be two distinct sites of a distributed system which maintain bank accounts A and B, respectively. A site refers to a process in this example. Let the communication channels from site S_1 to site S_2 and from site S_2 to site S_1 be denoted by C_{12} and C_{21} , respectively. Consider the following sequence of actions, which are also illustrated in the timing diagram of Figure 6.3:
- Time t_0 : Initially, Account A=\$600, Account B=\$200, $C_{12} = \$0$, $C_{21} = \$0$.
- Time t_1 : Site S_1 initiates a transfer of \$50 from Account A to Account B.
- Account A is decremented by \$50 to \$550 and a request for \$50 credit to Account B is sent on Channel C_{12} to site S_2 . Account A=\$550, Account B=\$200, $C_{12} = \$50$, $C_{21} = \$0$.

- Time t_2 : Site S2 initiates a transfer of \$80 from Account B to Account A.
- Account B is decremented by \$80 to \$120 and a request for \$80 credit to Account A is sent on Channel C_{21} to site S1. Account A=\$550, Account B=\$120, $C_{12} = \$50$, $C_{21} = \$80$.
- Time t_3 : Site S1 receives the message for a \$80 credit to Account A and updates Account A.
Account A=\$630, Account B=\$120, $C_{12} = \$50$, $C_{21} = \$0$.
- Time t_4 : Site S2 receives the message for a \$50 credit to Account B and updates Account B.
Account A=\$630, Account B=\$170, $C_{12} = \$0$, $C_{21} = \$0$.

T_3 : Site S1 receives the message for a \$80 credit to Account A and updates



T_4 : Site S2 receives the message for a \$50 credit to Account B and updates Account B

Figure 6.3: Money Transfer Example

- Suppose the local state of Account A is recorded at time t_0 to show \$600 and the local state of Account B and channels C_{12} and C_{21} are recorded at time t_2 to show \$120, \$50, and \$80, respectively. Then the recorded global state shows \$850 in the system. An extra \$50 appears in the system.
- **The reason for the inconsistency** is that Account A's state was recorded before the \$50 transfer to Account B using channel C_{12} was initiated, whereas channel C_{12} 's state was recorded after the \$50 transfer was initiated.
- This simple example shows that recording a consistent global state of a distributed system is not a trivial task. Recording activities of individual components must be coordinated appropriately.

Model of Communication

- Recall, there are three models of communication: FIFO, non-FIFO, and Co.
- In **FIFO model**, each channel acts as a first-in first-out message queue and thus, message ordering is preserved by a channel.
- In **non-FIFO model**, a channel acts like a set in which the sender process adds messages and the receiver process removes messages from it in a random order.
- A system that supports **causal delivery** of messages satisfies the following property: “For any two messages m_{ij} and m_{kj} ,
if $send(m_{ij}) \rightarrow send(m_{kj})$, then $rec(m_{ij}) \rightarrow rec(m_{kj})$ ”

Snapshot algorithm for FIFO channels

Chandy-Lamport algorithm:

- The **Chandy-Lamport** algorithm uses a **control message**, called a **marker** whose role in a **FIFO system** is to separate messages in the channels.
- After a site has recorded its snapshot, it sends a **marker**, along all of its outgoing channels before sending out any more messages.
- A marker separates the messages in the channel into those to be included in the snapshot from those not to be recorded in the snapshot.
- A process must record its snapshot no later than when it receives a marker on any of its incoming channels.

Chandy-Lamport Algorithm

- The algorithm can be initiated by any process by executing the “**Marker Sending Rule**” by which it records its local state and sends a marker on each outgoing channel.
- A process executes the “**Marker Receiving Rule**” on receiving a marker. If the process has not yet recorded its local state, it records the state of the channel on which the marker is received as empty and executes the “Marker Sending Rule” to record its local state.
- The algorithm terminates after each process has received a marker on all of its incoming channels.
- All the local snapshots get disseminated to all other processes and all the processes can determine the global state.

Chandy-Lamport Algorithm

Marker Sending Rule for process i

- 1) Process i records its state.
- 2) For each outgoing channel C on which a marker has not been sent, i sends a marker along C before i sends further messages along C .

Marker Receiving Rule for process j

On receiving a marker along channel C :

if j has not recorded its state **then**

Record the state of C as the empty set

Follow the “Marker Sending Rule”

else

Record the state of C as the set of messages received along C after j 's state was recorded and before j received the marker along C

Correctness and Complexity

Correctness

- Due to FIFO property of channels, it follows that no message sent after the marker on that channel is recorded in the channel state. Thus, condition **C2** is satisfied.
- When a process p_j receives message m_{ij} that precedes the marker on channel C_{ij} , it acts as follows: If process p_j has not taken its snapshot yet, then it includes m_{ij} in its recorded snapshot. Otherwise, it records m_{ij} in the state of the channel C_{ij} . Thus, condition **C1** is satisfied.

Complexity

- The recording part of a single instance of the algorithm **requires $O(e)$ messages and $O(d)$ time**, where e is the number of **edges** in the network and d is the **diameter** of the network.

Properties of the recorded global state

- The recorded global state may not correspond to any of the global states that occurred during the computation.
- Consider two possible executions of the snapshot algorithm (shown in Figure 6.4) for the previous money transfer example .

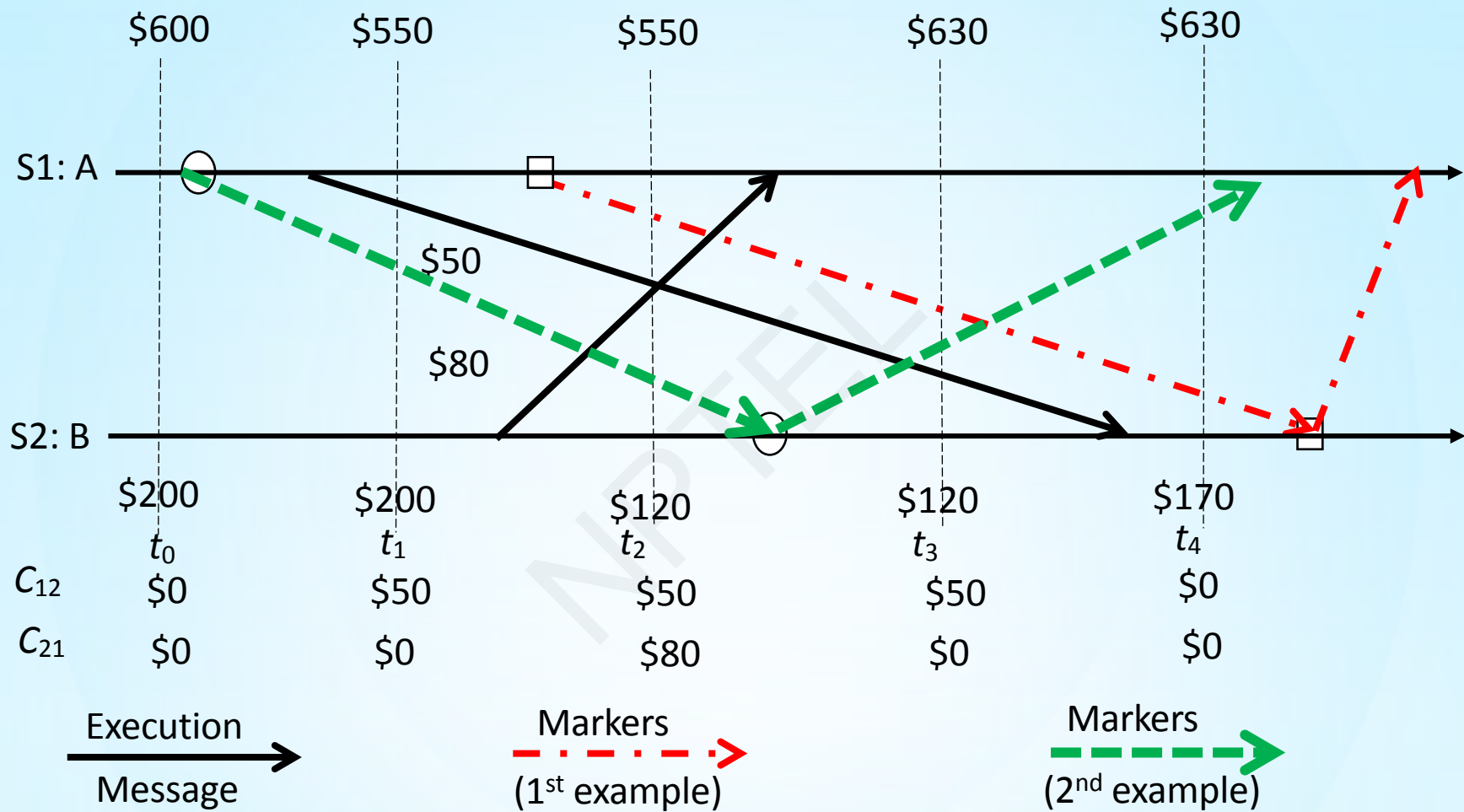


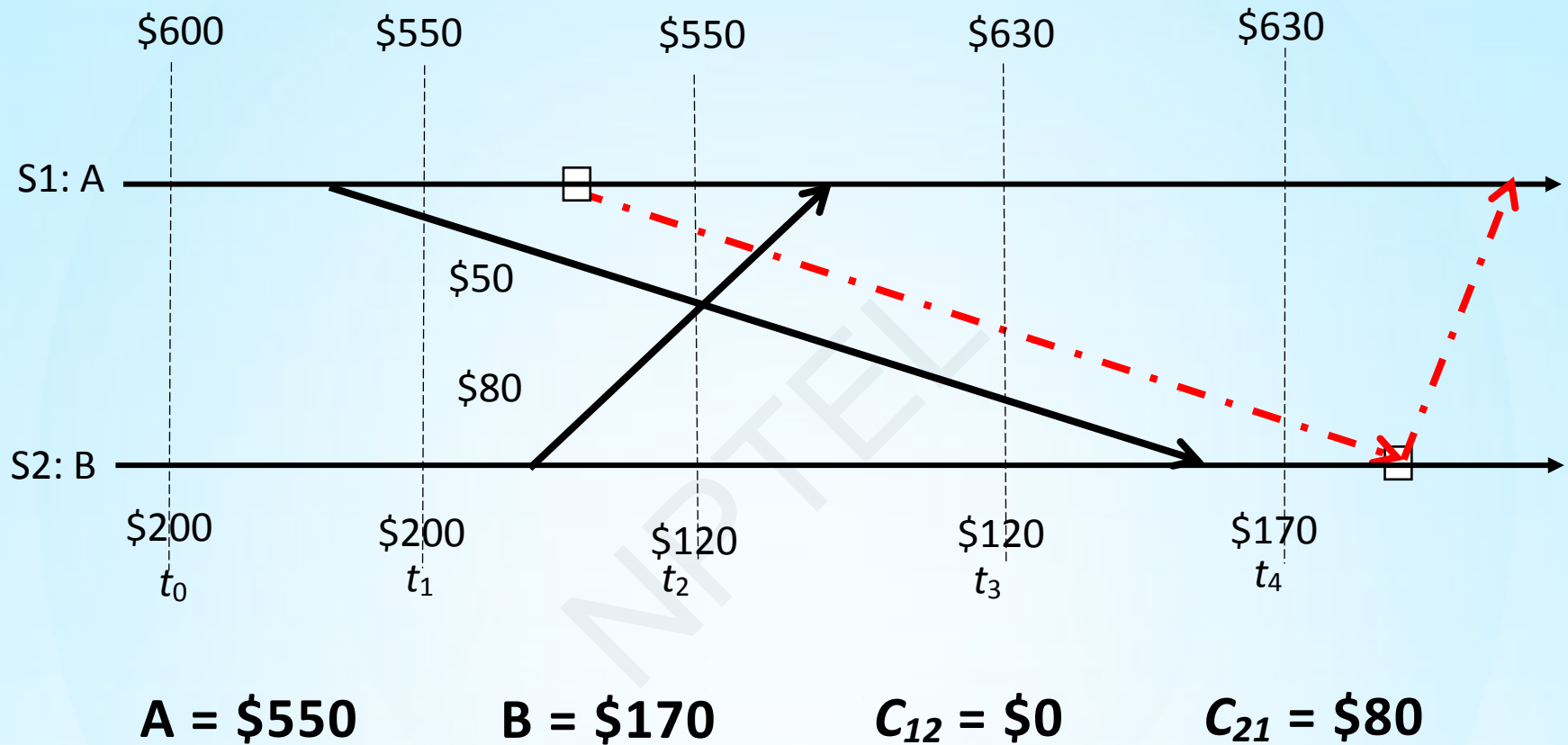
Figure 6.4: Timing diagram of two possible executions of the banking example

Properties of the recorded global state

1. (Markers shown using red dashed-and-dotted arrows.)

Let site S1 initiate the algorithm just after t_1 . Site S1 records its local state (account A=\$550) and sends a marker to site S2. The marker is received by site S2 after t_4 . When site S2 receives the marker, it records its local state (account B=\$170), the state of channel C_{12} as \$0, and sends a marker along channel C_{21} . When site S1 receives this marker, it records the state of channel C_{21} as \$80. The \$800 amount in the system is conserved in the recorded global state,

$$A = \$550, B = \$170, C_{12} = \$0, C_{21} = \$80$$



The \$800 amount in the system is conserved in the recorded global state

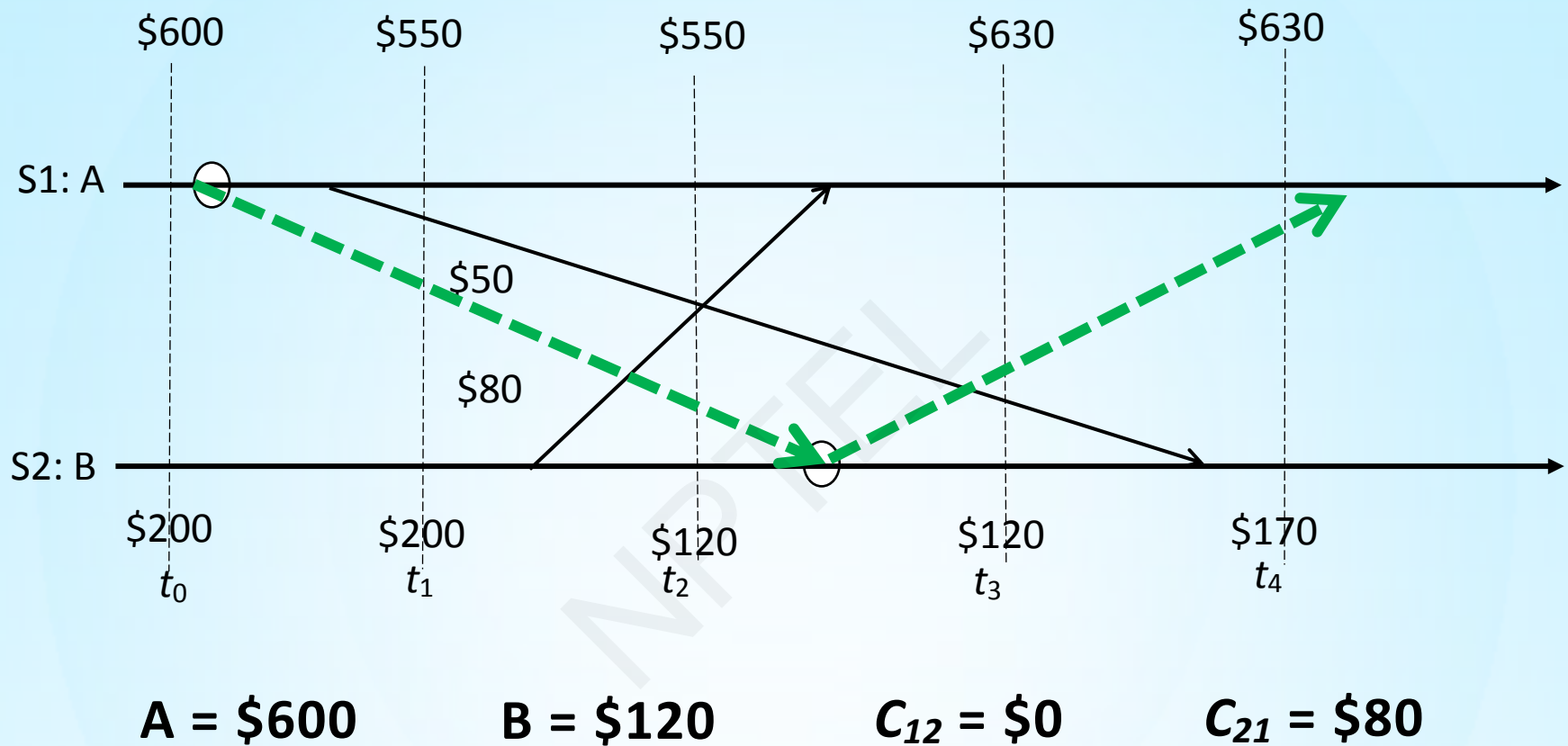
Figure 6.4: Timing diagram of two possible executions of the banking example

Properties of the recorded global state

2. (Markers shown using green dotted arrows.)

Let site S1 initiate the algorithm just after t_0 and before sending the \$50 for S2. Site S1 records its local state (account A = \$600) and sends a marker to site S2. The marker is received by site S2 between t_2 and t_3 . When site S2 receives the marker, it records its local state (account B = \$120), the state of channel C_{12} as \$0, and sends a marker along channel C_{21} . When site S1 receives this marker, it records the state of channel C_{21} as \$80. The \$800 amount in the system is conserved in the recorded global state,

$$A = \$600, B = \$120, C_{12} = \$0, C_{21} = \$80$$



The \$800 amount in the system is conserved in the recorded global state

Figure 6.4: Timing diagram of two possible executions of the banking example

Properties of the recorded global state

- In both these possible runs of the algorithm, the recorded global states never occurred in the execution.
- This happens because a process can change its state asynchronously before the markers it sent are received by other sites and the other sites record their states.
 - But the system could have passed through the recorded global states in some equivalent executions.
 - The recorded global state is a valid state in an equivalent execution and if a stable property (i.e., a property that persists) holds in the system before the snapshot algorithm begins, it holds in the recorded global snapshot.
- Therefore, a recorded global state is useful in detecting stable properties.

Variants of Global Snapshot Algorithms

Algorithms	Features
Chandy- Lamport	Baseline algorithm. Requires FIFO channels. $O(e)$ messages to record snapshot and $O(d)$ time.
Spezialetti- Kearns	supports concurrent initiators, efficient assembly and distribution of a snapshot. Assumes bidirectional channels. $O(e)$ messages to record, $O(rn^2)$ messages to assemble and distribute snapshot.
Lai-Yang	Works for non-FIFO channels. Markers piggybacked on computation messages. Message history required to compute channel states.
Li et al.	Small message history needed as channel states are computed incrementally.
Mattern	No message history required. Termination detection required to compute channel states.
Acharya- Badrinath	Requires causal delivery support, Centralized computation of channel states, Channel message contents need not be known. Requires $2n$ messages, 2 time units.
Alagar-Venkatesan	Requires causal delivery support, Distributed computation of channel states. Requires $3n$ messages, 3 time units, small messages.

**n = # processes, u = # edges on which messages were sent after previous snapshot,
 e = # channels, d is the diameter of the network, r = # concurrent initiators.**

Conclusion

- Recording global state of a distributed system is an important paradigm in the design of the distributed systems and the design of efficient methods of recording the global state is an important issue.
- This lecture first discussed a formal definition of the **global state of a distributed system and issues** related to its capture; then we have discussed the **Chandy-Lamport Algorithm** to record a snapshot of a distributed system.
- In upcoming lecture, we will discuss about distributed mutual exclusion algorithms.

Lecture: 07

Distributed Mutual Exclusion Algorithms & Non-Token Based Approaches



Rajiv Misra

Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have discussed about the **Global states** (i.e. **consistent, strongly consistent & inconsistent**), **Cuts in the space time diagram**, **Models of communication** and
- **Snapshot algorithm** i.e. **Chandy-Lamport algorithm** to record the global snapshot.

Preface

Content of this Lecture:

- In this lecture, we will discuss about **Mutual exclusion algorithms** for **distributed computing systems** such as:
 - (i) Non-token-based approach,
 - (ii) Quorum based approach and
 - (iii) Token-based approach

Introduction

- **Mutual exclusion:** Concurrent access of processes to a shared resource or data is executed in mutually exclusive manner.
- Only one process is allowed to execute the **critical section (CS)** at any given time.
- In a distributed system, shared variables (**semaphores**) or a local kernel cannot be used to implement mutual exclusion.
- Message passing is the sole means for implementing distributed mutual exclusion.

Approaches of Distributed Mutual Exclusion

- Distributed mutual exclusion algorithms must deal with **unpredictable message delays** and **incomplete knowledge** of the **system state**.
- Three basic approaches for distributed mutual exclusion are:
 - i. **Non-Token based approach**
 - ii. **Quorum based approach**
 - iii. **Token based approach**

(i) Non-token based approach:

- Two or more successive **rounds of messages** are exchanged among the sites to determine which site will enter the CS next.

e.g. Lamport Algorithm, RicartAgarwala Algorithm etc.

(ii) Quorum based approach:

- Each site requests permission to execute the CS from a subset of sites (called a **quorum**).
- Any two quorums contain a **common site**.
- This common site is responsible to make sure that only one request executes the CS at any time.
- e.g. Maekawa's Algorithm, Agarwal-El Abbadi Quorum-Based Algorithm etc.

(iii) Token-based approach:

- A unique token (also known as the **PRIVILEGE message**) is shared among the sites.
- A site is allowed to enter its CS if it possesses the token.
- Mutual exclusion is ensured because the **token is unique**.
- e.g. Suzuki-Kasami's Broadcast Algorithm, Raymond's Tree-Based Algorithm etc.

Preliminaries: System Model

- The system consists of N sites, $S_1, S_2, \dots, S_N$.
- We assume that a single process is running on each site. The process at site S_i is denoted by p_i .
- A site can be in one of the following three states: requesting the CS, executing the CS, or neither requesting nor executing the CS (i.e., idle).
- In the 'requesting the CS' state, the site is blocked and can not make further requests for the CS. In the 'idle' state, the site is executing outside the CS.
- In token-based algorithms, a site can also be in a state where a site holding the token is executing outside the CS (called the **idle token** state).
- At any instant, a site may have several pending requests for CS. A site queues up these requests and serves them one at a time.

Requirements of Mutual Exclusion Algorithms

- 1. Safety Property:** At any instant, **only one process** can execute the critical section.
- 2. Liveness Property:** This property states the absence of **deadlock and starvation**. Two or more sites should not endlessly wait for messages which will never arrive.
- 3. Fairness:** Each process gets a fair chance to execute the CS. Fairness property generally means the CS execution requests are **executed in the order of their arrival** (time is determined by a **logical clock**) in the system.

Performance Metrics

The performance is generally measured by the following four metrics:

(i) Message complexity: The **number of messages** required per CS execution by a site.

(ii) Synchronization delay: After a site leaves the CS, it is the time required and before the next site enters the CS (see Figure 7.1).

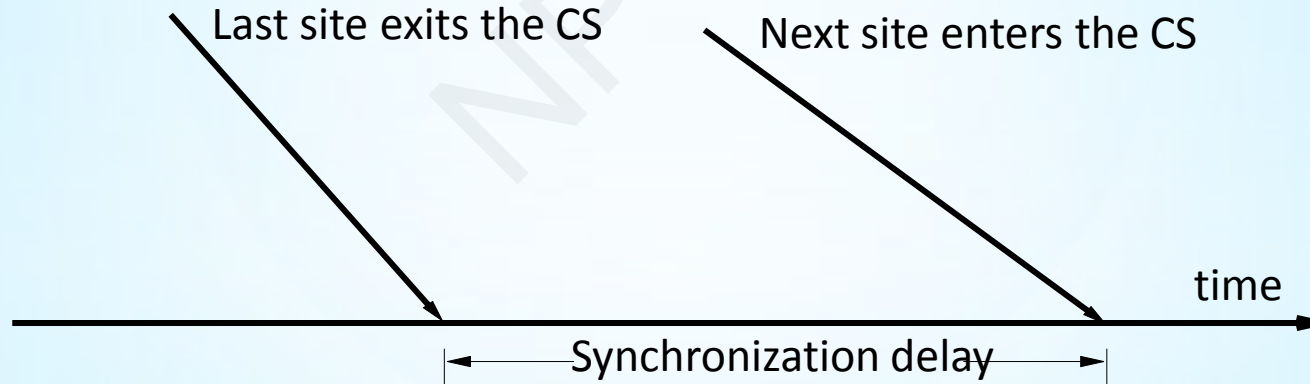
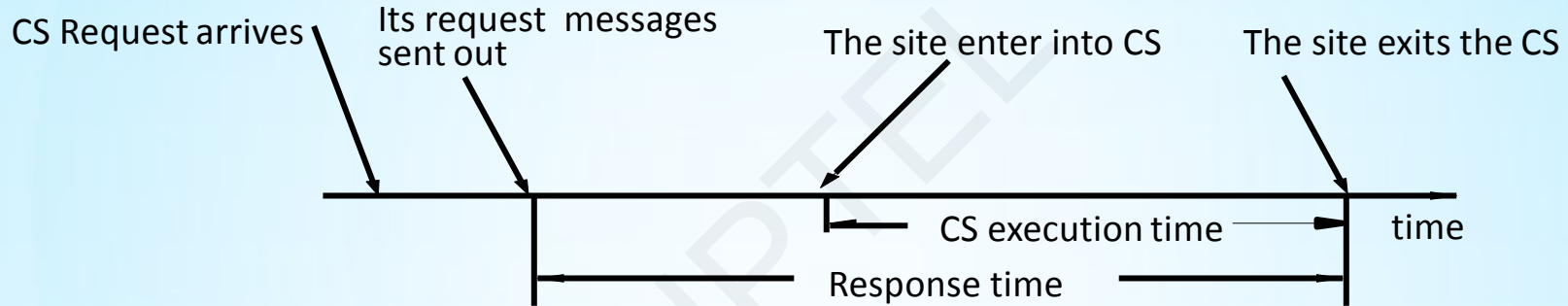


Figure 7.1: Synchronization Delay

Performance Metrics

(iii) Response time: The time interval a request waits for its CS execution to be over after its request messages have been sent out (see below Figure 7.2)



(iv) System throughput: The rate at which the system executes requests for the CS.
$$\text{system throughput} = 1 / (SD + E)$$

where SD is the synchronization delay and E is the average critical section execution time.

Performance Metrics

Low and High Load Performance:

- We often study the performance of mutual exclusion algorithms under two special loading conditions, viz., “*low load*” and “*high load*”.
- The load is determined by the arrival rate of CS execution requests.
- Under *low load* conditions, there is seldom more than one request for the critical section present in the system simultaneously.
- Under *heavy load* conditions, there is always a pending request for critical section at a site.

(i) Non-token based Approaches

(i) Lamport's Algorithm

- Requests for CS are executed in the increasing order of timestamps and time is determined by logical clocks.
- Every site S_i keeps a queue, **request_queue_i**, which contains mutual exclusion requests ordered by their timestamps.
- This algorithm requires communication channels to deliver messages the **FIFO order**. Three types of messages are used- **Request, Reply and Release**. These messages with timestamps also **updates** logical clock.

The Algorithm

Requesting the critical section:

- When a site S_i wants to enter the CS, it broadcasts a **REQUEST**(ts_i, i) message to all other sites and places the request on *request_queue_i*.
((ts_i, i) denotes the timestamp of the request.)
- When S_j receives the **REQUEST**(ts_i, i) message from site S_i , S_j places site S_i 's request on *request_queue_j* and it returns a *timestamped* **REPLY** message to S_i .

Executing the critical section: Site S_i enters the CS when the following two conditions hold:

- L1:** S_i has received a **message** with timestamp larger than (ts_i, i) from all other sites.
- L2:** S_i 's request is at the top of *request_queue_i*.

The Algorithm

Releasing the critical section:

- Site S_i , upon exiting the CS, removes its request from the top of its request queue and broadcasts a *timestamped* **RELEASE** message to all other sites.
- When a site S_j receives a **RELEASE** message from site S_i , it removes S_i 's request from its request queue.

When a site removes a request from its request queue, its own request may come at the top of the queue, enabling it to enter the CS.

Correctness

Theorem: Lamport's algorithm achieves mutual exclusion.

Proof:

- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently. For this to happen conditions L1 and L2 must hold at both the sites *concurrently*.
- This implies that at some instant in time, say t , both S_i and S_j have their own requests at the top of their *request_queues* and condition L1 holds at them. Without loss of generality, assume that S_i 's request has smaller timestamp than the request of S_j .
- From condition L1 and FIFO property of the communication channels, it is clear that at instant t the request of S_i must be present in *request_queue_j* when S_j was executing its CS. This implies that S_j 's own request is at the top of its own *request_queue* when a smaller timestamp request, S_i 's request, is present in the *request_queue_j* – a contradiction!

Correctness

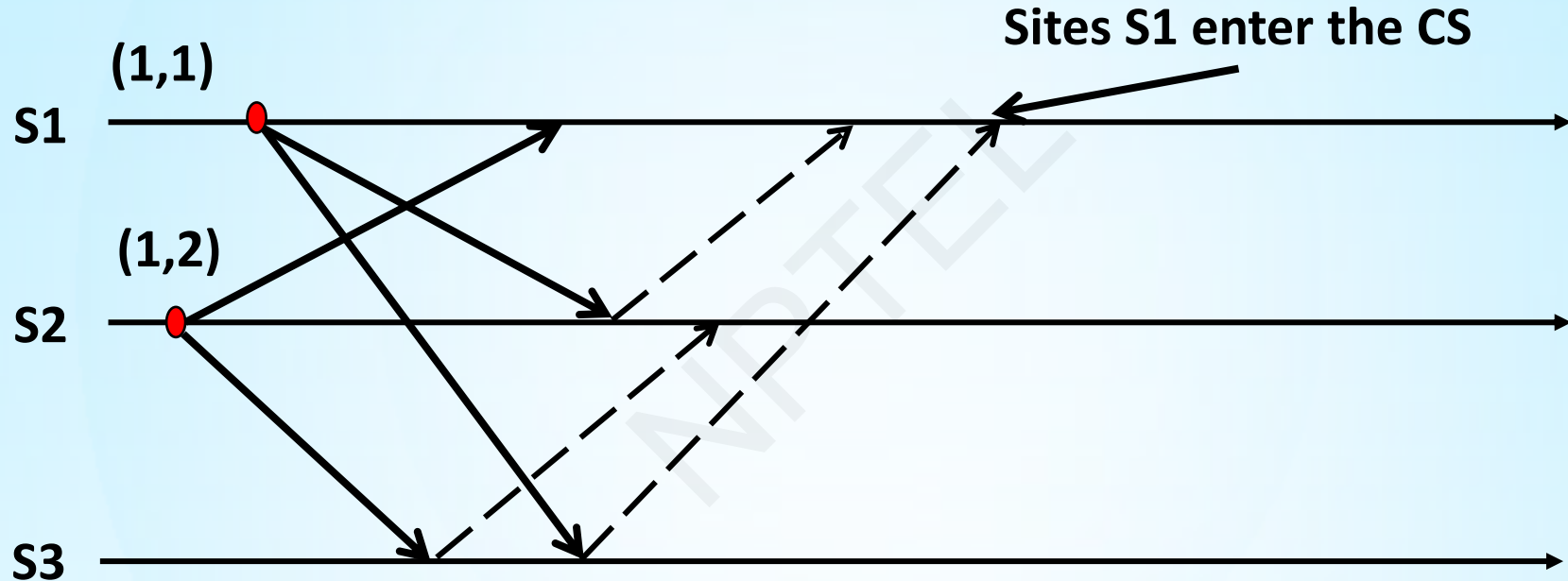
Theorem: Lamport's algorithm is fair.

Proof:

- The proof is by contradiction. Suppose a site S_i 's request has a smaller timestamp than the request of another site S_j and S_j is able to execute the CS before S_i .
- For S_j to execute the CS, it has to satisfy the conditions **L1** and **L2**. This implies that at some instant in time say t , S_j has its own request at the top of its queue and it has also received a message with timestamp larger than the timestamp of its request from all other sites.
- But *request_queue* at a site is ordered by timestamp, and according to our assumption S_i has lower timestamp. So S_i 's request must be placed ahead of the S_j 's request in the *request_queue_j*. This is a contradiction!

Lamport's Algorithm Example:

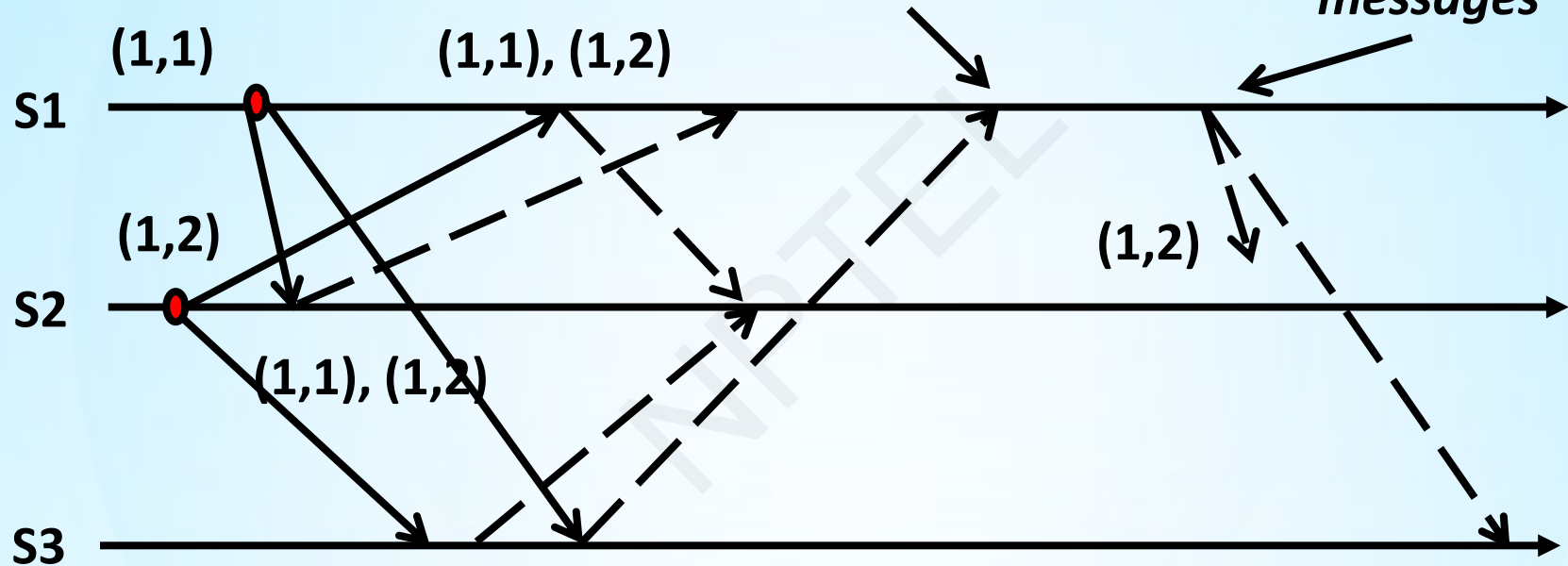
Sites S1 and S2 are Making Requests for the CS



Lamport's Algorithm Example:

Site S1 enters the CS

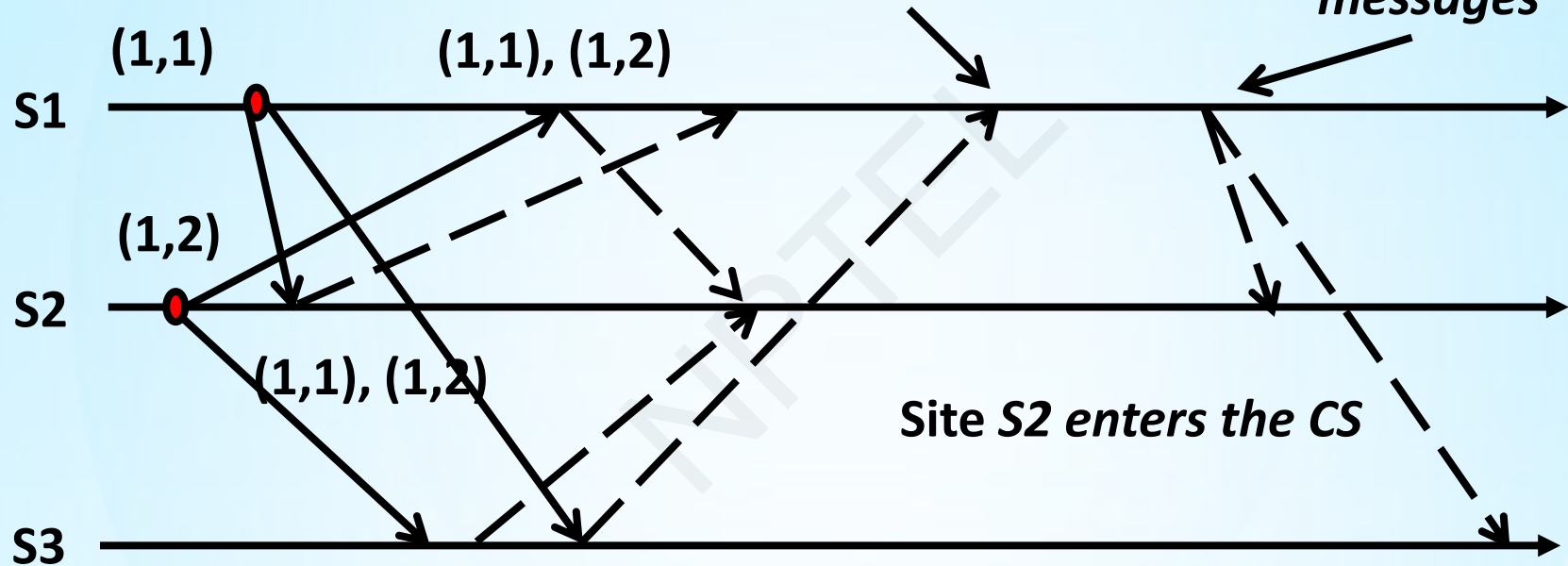
*Site S1 exits the CS
and sends RELEASE
messages*



Lamport's Algorithm Example:

Site S1 enters the CS

*Site S1 exits the CS
and sends RELEASE
messages*



Performance

- For each CS execution, Lamport's algorithm requires $(N - 1)$ **REQUEST** messages, $(N - 1)$ **REPLY** messages, and $(N - 1)$ **RELEASE** messages.
- Thus, Lamport's algorithm requires $3(N - 1)$ messages per CS invocation.
- Synchronization delay in the algorithm is T .

An Optimization

- In Lamport's algorithm, **REPLY** messages can be omitted in certain situations. For example, if site S_j receives a **REQUEST** message from site S_i after it has sent its own **REQUEST** message with timestamp higher than the timestamp of site S_i 's request, then site S_j need not send a **REPLY** message to site S_i .
- This is because when site S_j receives site S_i 's request with timestamp higher than its own, it can conclude that site S_j does not have any smaller timestamp request which is still pending.
- With this optimization, Lamport's algorithm requires **between $3(N - 1)$ and $2(N - 1)$ messages per CS execution.**

(ii) Ricart-Agrawala Algorithm

- The Ricart-Agrawala algorithm assumes the communication channels are **FIFO**. The algorithm uses two types of messages: **REQUEST** and **REPLY**.
- A process sends a **REQUEST** message to all other processes to request their permission to enter the critical section. A process sends a **REPLY** message to a process to give its permission to that process.
- Processes use **Lamport-style logical clocks** to assign a timestamp to critical section requests and timestamps are used to decide the priority of requests.
- Each process p_i maintains the **Request-Deferred array**, RD_i , the size of which is the same as the number of processes in the system.
- Initially, $\forall i \ \forall j: RD_i[j]=0$. Whenever p_i defer the request sent by p_j , it sets $RD_i[j]=1$ and after it has sent a REPLY message to p_j , it sets $RD_i[j]=0$.

Description of the Algorithm

Requesting the critical section:

- (a) When a site S_i wants to enter the CS, it broadcasts a timestamped **REQUEST** message to all other sites.
- (b) When site S_j receives a **REQUEST** message from site S_i , it sends a **REPLY** message to site S_i if site S_j is neither requesting nor executing the CS, or if the site S_j is requesting and S_i 's request's timestamp is smaller than site S_j 's own request's timestamp. Otherwise, the reply is deferred and S_j sets $RD_j[i]=1$

Executing the critical section:

- (c) Site S_i enters the CS after it has received a **REPLY** message from every site it sent a **REQUEST** message to.

Contd...

Releasing the critical section:

(d) When site S_i exits the CS, it sends all the deferred **REPLY** messages: $\forall j$ if $RD_i[j]=1$, then send a **REPLY** message to S_j and set $RD_i[j]=0$.

Notes:

- When a site receives a message, it updates its clock using the timestamp in the message.
- When a site takes up a request for the CS for processing, it updates its local clock and assigns a timestamp to the request.

Correctness

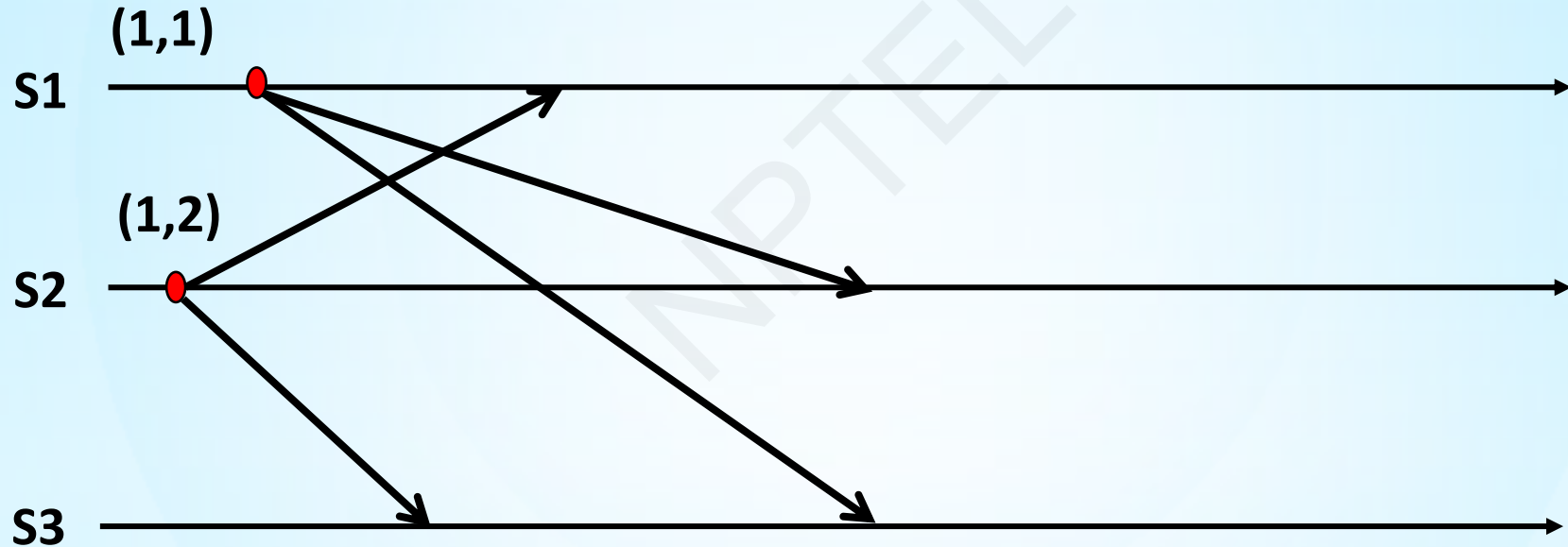
Theorem: Ricart-Agrawala algorithm achieves mutual exclusion.

Proof:

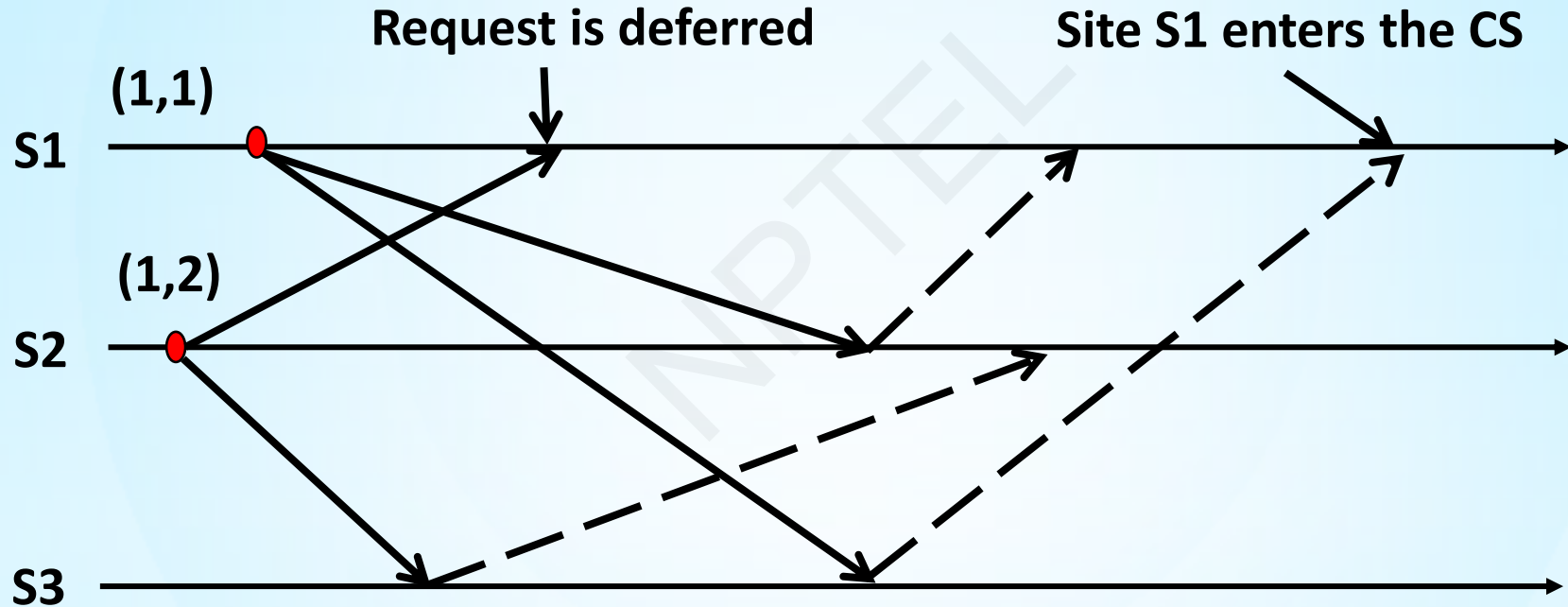
- Proof is by contradiction. Suppose two sites S_i and S_j are executing the CS concurrently and S_i 's request has higher priority than the request of S_j . Clearly, S_i received S_j 's request after it has made its own request.
- Thus, S_j can concurrently execute the CS with S_i only if S_i returns a **REPLY** to S_j (in response to S_j 's request) before S_i exits the CS.
- However, this is impossible because S_j 's request has lower priority.
- Therefore, Ricart-Agrawala algorithm achieves mutual exclusion.

Ricart-Agrawala algorithm Example:

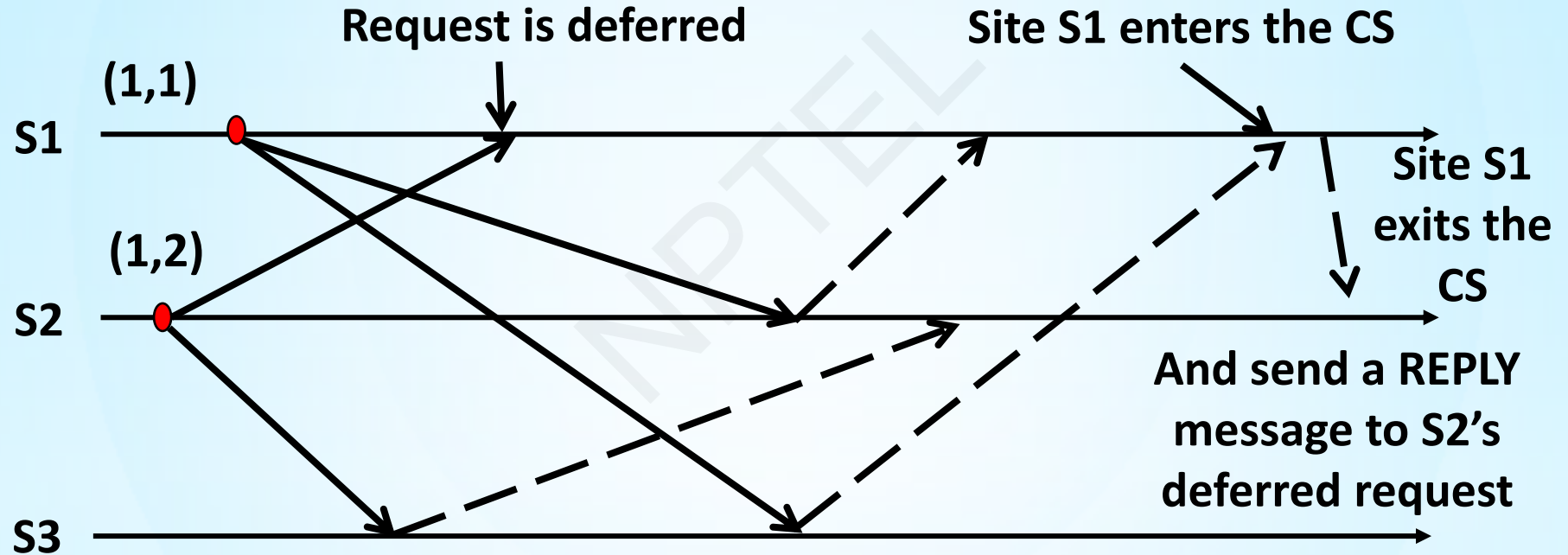
Sites S1 and S2 are Making Requests for the CS



Ricart-Agrawala algorithm Example:



Ricart-Agrawala algorithm Example:



Performance

- For each CS execution, Ricart-Agrawala algorithm requires $(N - 1)$ **REQUEST** messages and $(N - 1)$ **REPLY** messages.
- Thus, it requires $2(N - 1)$ **messages** per CS execution.
- Synchronization delay in the algorithm is T .

Conclusion

- **Mutual exclusion** is a fundamental problem in distributed computing systems, where concurrent access to a shared resource or data is serialized. Mutual exclusion in a distributed system requires that only one process be allowed to execute the critical section at any given time.
- In this lecture, we have discussed about the '**Concepts of Distributed Mutual Exclusion**' and its **Non-token based approaches** i.e. **Lamports algorithm** and **Ricart-Agrawala algorithm**
- In upcoming lectures, we will discuss about **Quorum based approaches** and **Token-based approaches**.

Lecture: 08

Quorum Based Distributed Mutual Exclusion Algorithms



Rajiv Misra

Dept. of Computer Science & Engineering
Indian Institute of Technology Patna
rajivm@iitp.ac.in

Preface

Recap of Previous Lecture:

- In the previous lecture, we have presented the **Concept of Distributed Mutual Exclusion Algorithms** for distributed computing systems.
- We have also discussed about **Non-Token based approaches** *i.e.* **Lamports algorithm** and **Ricart-Agrawala algorithm**

Preface

Content of this Lecture:

- In this lecture, we will discuss about the **Quorum based approaches** and
- Also discuss its few approaches namely **Maekawa's Algorithm** and **Agarwal-El Abbadi Quorum-Based Algorithm**

(ii) Quorum Based Approaches

Introduction

- In the '**quorum-based approach**', each site requests permission to execute the CS from a **subset of sites** (called a **quorum**).
- The quorums are formed in such a way that when two sites concurrently request access to the CS, at least one site receives both the requests and this site is responsible to make sure that only one request executes the CS at any time.

Quorum-Based Mutual Exclusion Algorithms

Quorum-based mutual exclusion algorithms are different in the following two ways:

1. A site does not request permission from all other sites, but only from a subset of the sites. The **request set** of sites are chosen such that $\forall i \forall j : 1 \leq i, j \leq N : R_i \cap R_j \neq \Phi$. Consequently, every pair of sites has a site which mediates conflicts between that pair.
2. A site can send out only **one REPLY** message at any time. A site can send a **REPLY** message only after it has received a **RELEASE** message for the previous **REPLY** message.

Contd...

Since these algorithms are based on the notion of '**Coteries**' and '**Quorums**', we next describe the idea of coteries and quorums.

A **coterie C** is defined as a set of sets, where each set $g \in C$ is called a **quorum**. The following properties hold for quorums in a coterie:

- **Intersection property:** For every quorum $g, h \in C$, $g \cap h \neq \emptyset$.
For example, sets $\{1,2,3\}$, $\{2,5,7\}$ and $\{5,7,9\}$ cannot be quorums in a coterie because the first and third sets **do not have a common element**.
- **Minimality property:** There should be no quorums g, h in **coterie C** such that $g \supseteq h$. For example, sets $\{1,2,3\}$ and $\{1,3\}$ cannot be quorums in a coterie because the first set is a superset of the second.

Contd...

Coterie and quorums can be used to develop algorithms to ensure mutual exclusion in a distributed environment. A simple protocol works as follows:

- Let 'a' is a site in quorum 'A'. If 'a' wants to invoke mutual exclusion, it requests permission from all sites in its quorum 'A'.
- Every site does the same to invoke mutual exclusion. Due to the Intersection Property, quorum 'A' contains at least one site that is common to the quorum of every other site.
- These common sites send permission to only one site at any time. Thus, mutual exclusion is guaranteed.

Note that the **Minimality property ensures efficiency** rather than correctness.

(i) Maekawa's Algorithm

- Maekawa's algorithm was the **first quorum-based mutual exclusion algorithm**. The **request sets for sites** (i.e., **quorums**) in Maekawa's algorithm are constructed to satisfy the following conditions:

M1: $(\forall i \forall j : i \neq j, 1 \leq i, j \leq N :: R_i \cap R_j \neq \phi)$

M2: $(\forall i : 1 \leq i \leq N :: S_i \in R_i)$

M3: $(\forall i : 1 \leq i \leq N :: |R_i| = K)$

M4: Any site S_j is contained in K number of R_i s, $1 \leq i, j \leq N$.

Maekawa used the theory of projective planes and showed that $N = K(K - 1) + 1$. This relation gives $|R_i| = \sqrt{N}$

Maekawa's Algorithm

- Conditions **M1** and **M2** are necessary for correctness; whereas conditions **M3** and **M4** provide other desirable features to the algorithm.
- Condition **M3** states that the size of the requests sets of all sites must be equal implying that all sites should have to do equal amount of work to invoke mutual exclusion.
- Condition **M4** enforces that exactly the same number of sites should request permission from any site implying that all sites have **“equal responsibility”** in granting permission to other sites.

The Algorithm

A site S_i executes the following steps to execute the CS.

Requesting the critical section

- (a) A site S_i requests access to the CS by sending **REQUEST(i)** messages to all sites in its request set R_i .
- (b) When a site S_j receives the **REQUEST(i)** message, it sends a **REPLY(j)** message to S_i provided it hasn't sent a REPLY message to a site since its receipt of the last **RELEASE** message. Otherwise, it queues up the **REQUEST(i)** for later consideration.

Executing the critical section

- (c) Site S_i executes the CS only after it has received a **REPLY** message from every site in R_i .

The Algorithm

Releasing the critical section

(d) After the execution of the CS is over, site S_i sends a **RELEASE(i)** message to every site in R_i .

(e) When a site S_j receives a **RELEASE(i)** message from site S_i , it sends a **REPLY** message to the next site waiting in the queue and deletes that entry from the queue. If the queue is empty, then the site updates its state to reflect that it has not sent out any **REPLY** message since the receipt of the last **RELEASE** message.

Correctness

Theorem: *Maekawa's algorithm achieves mutual exclusion.*

Proof:

- Proof is by contradiction. Suppose two sites S_i and S_j are concurrently executing the CS.
- This means site S_i received a **REPLY** message from all sites in R_i and concurrently site S_j was able to receive a **REPLY** message from all sites in R_j .
- If $R_i \cap R_j = \{S_k\}$, then site S_k must have sent **REPLY** messages to both S_i and S_j concurrently, which is a contradiction.

Correctness

- Since the size of a request set is $\sqrt{N}$, an execution of the CS requires $\sqrt{N}$ REQUEST, $\sqrt{N}$ REPLY, and $\sqrt{N}$ RELEASE messages, resulting in $3\sqrt{N}$ messages per CS execution.
- **Synchronization delay** in this algorithm is $2T$. This is because after a site S_i exits the CS, it first releases all the sites in R_i and then one of those sites sends a REPLY message to the next site that executes the CS.

Problem of Deadlocks

- **Maekawa's algorithm can deadlock** because a site is exclusively locked by other sites and requests are not prioritized by their timestamps.
- Assume three sites **S_i , S_j , and S_k simultaneously invoke mutual exclusion.**
- Suppose $R_i \cap R_j = \{S_{ij}\}$, $R_j \cap R_k = \{S_{jk}\}$, and $R_k \cap R_i = \{S_{ki}\}$.

Consider the following scenario:

1. S_{ij} has been locked by S_i (forcing S_j to wait at S_{ij}).
 2. S_{jk} has been locked by S_j (forcing S_k to wait at S_{jk}).
 3. S_{ki} has been locked by S_k (forcing S_i to wait at S_{ki}).
- This state represents a deadlock involving sites S_i , S_j , and S_k .

Handling Deadlocks

- Maekawa's algorithm handles deadlocks by requiring a site to yield a lock if **the timestamp of its request is larger than the timestamp of some other request** waiting for the same lock.
- A site **suspects a deadlock** (and initiates message exchanges to resolve it) whenever a higher priority request arrives and waits at a site because the site has sent a **REPLY message to a lower priority request.**

Handling Deadlocks

Deadlock handling requires **three types of messages**:

FAILED: A FAILED message from site S_i to site S_j indicates that S_i can not grant S_j 's request because it has currently granted permission to a site with a higher priority request.

INQUIRE: An INQUIRE message from S_i to S_j indicates that S_i would like to find out from S_j if it has succeeded in locking all the sites in its request set.

YIELD: A YIELD message from site S_i to S_j indicates that S_i is returning the permission to S_j (to yield to a higher priority request at S_j).

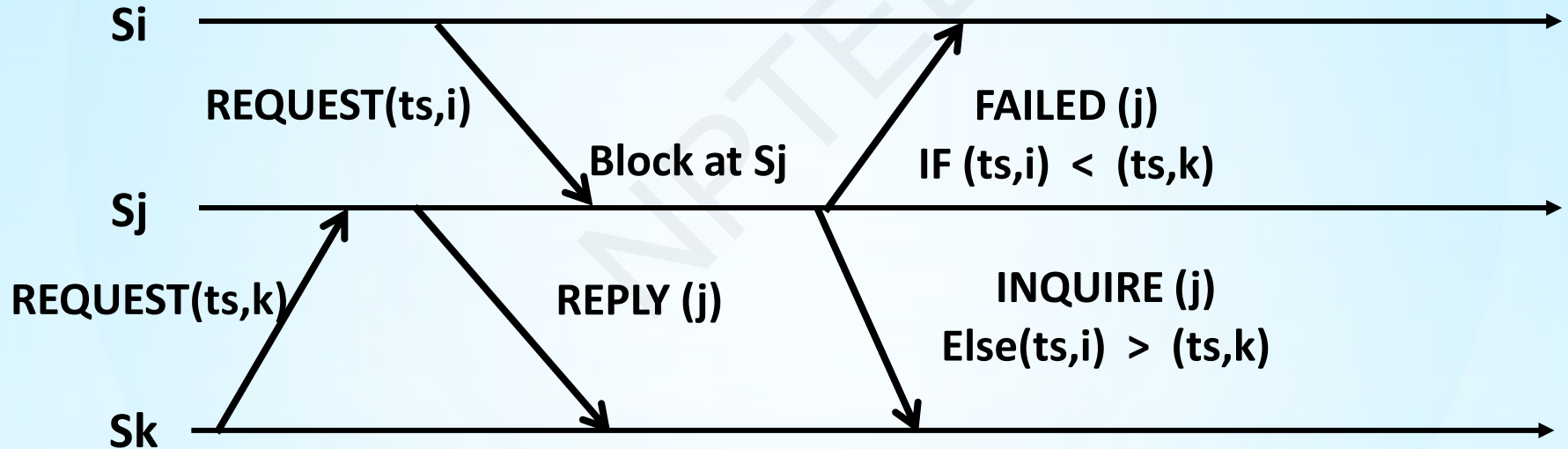
Handling Deadlocks

Maekawa's algorithm handles deadlocks as follows:

- When a **REQUEST**(ts, i) from site S_i blocks at site S_j because S_j has currently granted permission to site S_k , then S_j sends a **FAILED**(j) message to S_i if S_i 's request has lower priority. Otherwise, S_j sends an **INQUIRE**(j) message to site S_k .
- In response to an **INQUIRE**(j) message from site S_j , site S_k sends a **YIELD**(k) message to S_j provided S_k has received a **FAILED** message from a site in its request set or if it sent a **YIELD** to any of these sites, but has not received a new **GRANT** from it.
- In response to a **YIELD**(k) message from site S_k , site S_j assumes as if it has been released by S_k , places the request of S_k at appropriate location in the request queue, and sends a **GRANT**(j) to the top request's site in the queue. Maekawa's algorithm requires extra messages to handle deadlocks
- **Maximum number of messages** required per CS execution in this case is $5\sqrt{N}$

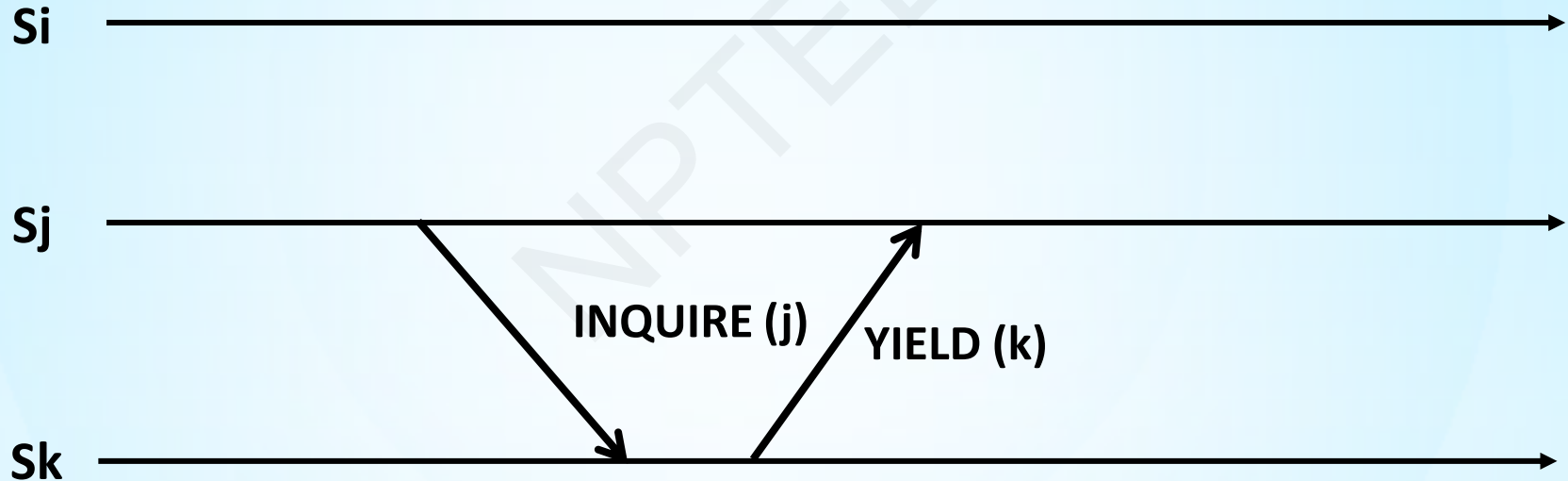
Handling Deadlocks: Case-I

When a **REQUEST**(ts, i) from site S_i blocks at site S_j because S_j has currently granted permission to site S_k , then S_j sends a **FAILED**(j) message to S_i if S_i 's request has lower priority. Otherwise, S_j sends an **INQUIRE**(j) message to site S_k .



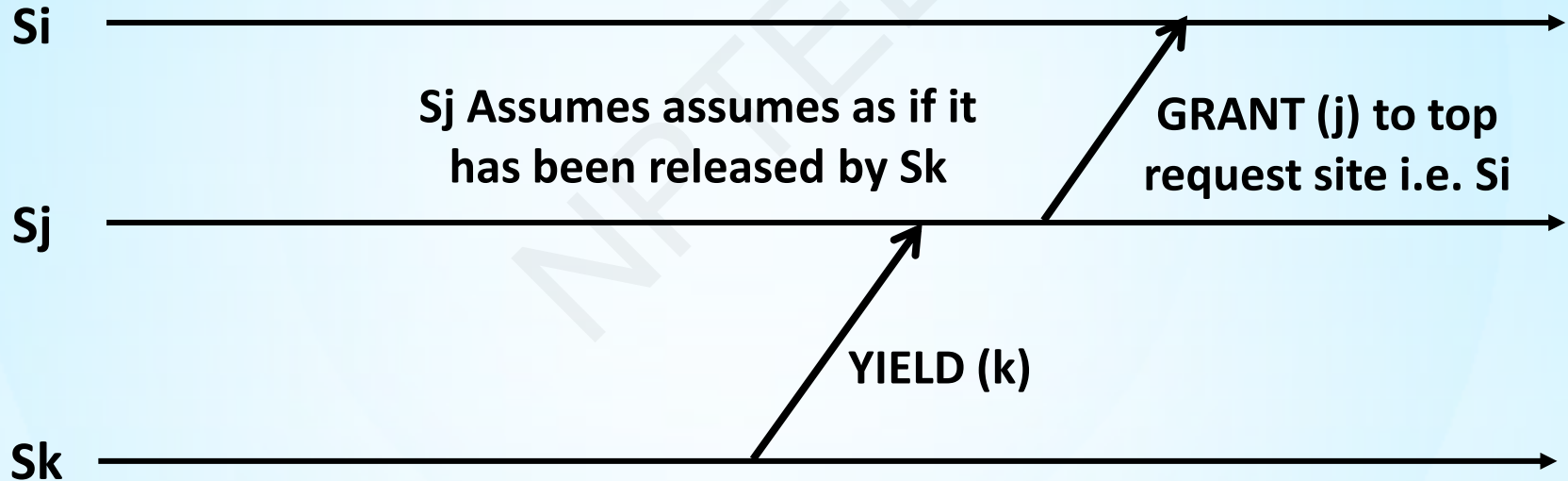
Handling Deadlocks: Case-II

In response to an **INQUIRE(j)** message from site S_j , site S_k sends a **YIELD(k)** message to S_j provided S_k has received a **FAILED** message from a site in its request set or if it sent a **YIELD** to any of these sites, but has not received a new **GRANT** from it.



Handling Deadlocks: Case-III

In response to a **YIELD(k)** message from site S_k , site S_j assumes as if it has been released by S_k , places the request of S_k at appropriate location in the request queue, and sends a **GRANT(j)** to the top request's site in the queue.



(ii) Agarwal-El Abbadi Quorum-Based Algorithm

- **Agarwal and El Abbadi** developed a simple and efficient mutual exclusion algorithm by introducing **tree quorums**.
- They gave a novel algorithm for constructing tree-structured quorums in the sense that it uses **hierarchical structure of a network**.
- The mutual exclusion algorithm is independent of the underlying topology of the network and there is **no need for a multicast facility** in the network.
- However, such facility will improve the performance of the algorithm.
- The mutual exclusion algorithm **assumes** that **sites in the distributed system can be organized into a structure such as tree, grid, binary tree, etc. and**
- There exists a **routing mechanism to exchange messages** between different sites in the system.

(ii) Agarwal-El Abbadi Quorum-Based Algorithm

- Agarwal-El Abbadi quorum-based algorithm uses '**tree-structured quorums**'
- All the sites in the system are logically organized into a **complete binary tree**.
- For a complete binary tree with **level 'k'**, we have **$2^{k+1} - 1$ sites** with its root at level k and leaves at level 0.
- The number of sites in a path from the root to a leaf is equal to the level of the tree k+1 which is equal to **$O(\log n)$** .
- A **path in a binary tree is the sequence** $a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_k$ such that a_i is the parent of a_{i+1} .

Algorithm for constructing a tree-structured quorum

```
FUNCTION GetQuorum (Tree: NetworkHierarchy): QuorumSet;  
  VAR left, right : QuorumSet;  
  BEGIN  
    IF Empty (Tree) THEN  
      RETURN ({});  
    ELSE IF GrantsPermission(Tree↑.Node) THEN  
      RETURN ((Tree↑.Node) ∪ GetQuorum (Tree↑.LeftChild));  
    OR  
      RETURN ((Tree↑.Node) ∪ GetQuorum (Tree↑.RightChild));(*line 9*)  
    ELSE  
      left←GetQuorum(Tree↑.left);  
      right←GetQuorum(Tree↑.right);  
      IF (left = ∅ ∨ right = ∅) THEN  
        (* Unsuccessful in establishing a quorum *)  
        EXIT(-1);  
      ELSE  
        RETURN (left ∪ right);  
      END; (* IF *)  
    END; (* IF *)  
  END; (* IF *)  
END GetQuorum
```

Algorithm for constructing a tree-structured quorum: Explanation

- The algorithm for constructing tree-structured quorums uses two functions called **GetQuorum(Tree)** and **GrantsPermission(site)** and assumes that there is a well-defined root for the tree.
- **GetQuorum** is a recursive function that takes a **tree node “x”** as the parameter and calls **GetQuorum** for its child node provided that the **GrantsPermission(x)** is true.
- The **GrantsPermission(x)** is **true** only when the **node “x”** agrees to be in the quorum. If the **node “x”** is down due to a failure, then it may not agree to be in the quorum and the value of **GrantsPermission(x)** will be **false**.

Algorithm for constructing a tree-structured quorum:

Explanation

- The algorithm tries to construct quorums in a way that each quorum represents **any path from the root to a leaf**. *i.e.*, in this case the (no failures) quorum is any set $a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_k$, where a_1 is the root and a_k is a leaf, and for all $i < k$, a_i is the parent of a_{i+1} .
- If it fails to find such a path (say, because node 'x' has failed), the control goes to the **ELSE** block which specifies that the failed node 'x' is substituted by two paths both of which start with the left and right children of 'x' and end at leaf nodes.
- If the leaf site is down or inaccessible due to any reason, then the quorum cannot be formed and the algorithm terminates with an error condition.
- The sets that are constructed using this algorithm are termed as **tree quorums**.

Analysis of the algorithm for constructing tree-structured quorums

- The **best case scenario** of the algorithm takes $O(\log n)$ sites to form a tree quorum. There are certain cases where even in the event of a failure, $O(\log n)$ sites are sufficient to form a tree quorum.
- For example, if the site that is parent of a leaf node fails, then the **number of sites that are necessary for a quorum will be still $O(\log n)$** . Thus, the algorithm requires very **few messages** in a relatively **fault-free environment**.
- It can tolerate the failure up to $n - O(\log n)$ sites and still form a tree quorum.
- **In the worst case**, the algorithm requires the majority of sites to construct a tree quorum and the number of sites is same for all cases (faults or no faults). The **worst case tree quorum size** is determined as $O((n+1)/2)$ by **induction**.

Examples of Tree-Structured Quorums

When there is no node failure, the number of quorums formed is equal to the number of leaf sites.

- Consider the tree of height 3 shown in Figure 8.1, constructed from 15 ($=2^{3+1}-1$) sites.
- In this case 8 quorums are formed from 8 possible root-leaf paths: 1-2-4-8, 1-2-4-9, 1-2-5-10, 1-2-5-11, 1-3-6-12, 1-3-6-13, 1-3-7-14 and 1-3-7-15.
- If any site fails, the algorithm substitutes for that site two possible paths starting from the site's two children and ending in leaf nodes.
- For example, when node 3 fails, we consider possible paths starting from children 6 and 7 and ending at leaf nodes. The possible paths starting from child 6 are 6-12 and 6-13, and from child 7 are 7-14 and 7-15.
- So, when node 3 fails, the following eight quorums can be formed:
 $\{1,6,12,7,14\}$, $\{1,6,12,7,15\}$, $\{1,6,13,7,14\}$, $\{1,6,13,7,15\}$, $\{1,2,4,8\}$,
 $\{1,2,4,9\}$, $\{1,2,5,10\}$, $\{1,2,5,11\}$.

8 quorums are

formed :

1-2-4-8,
1-2-4-9,
1-2-5-10,
1-2-5-11,
1-3-6-12,
1-3-6-13,
1-3-7-14 and
1-3-7-15

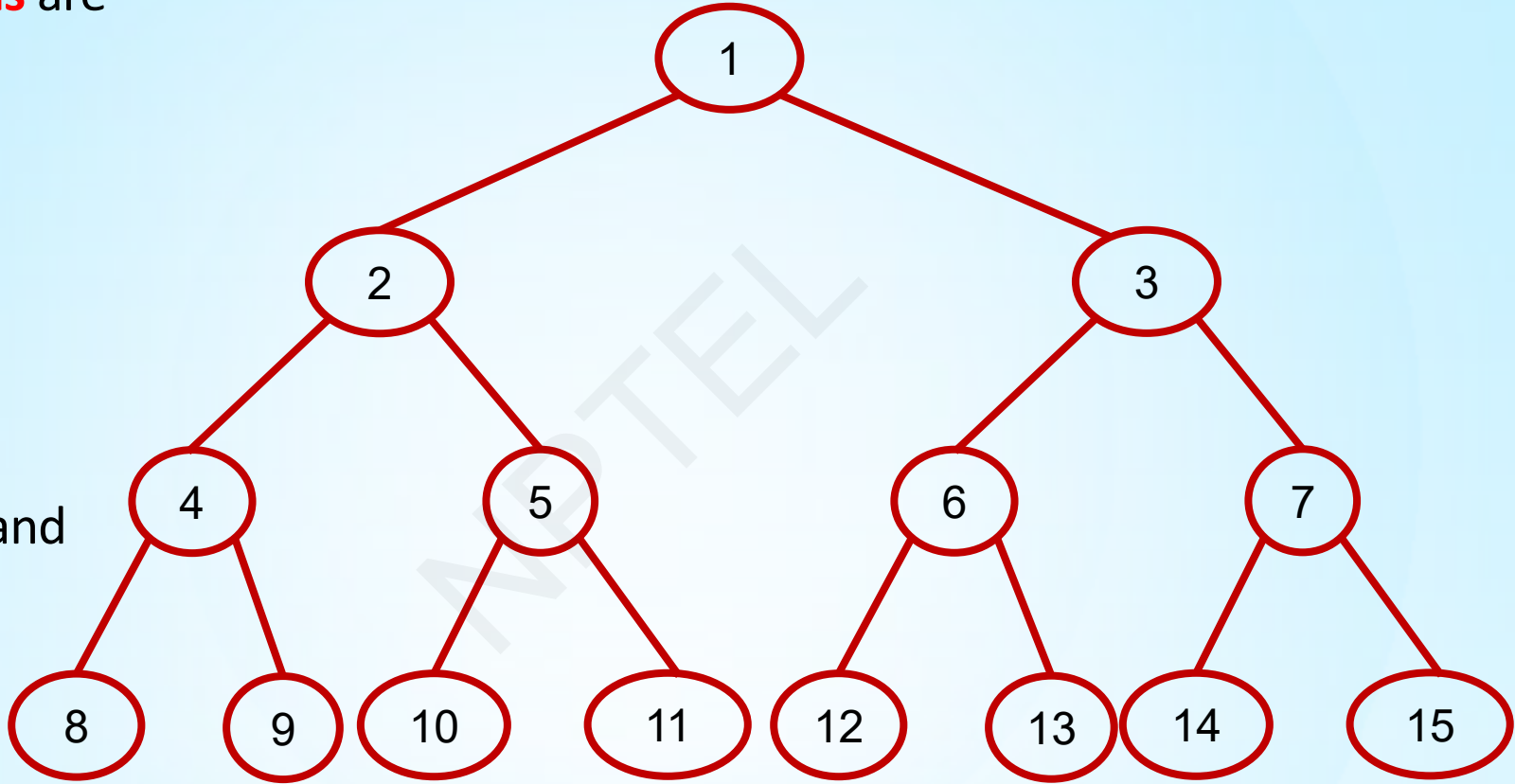


Figure 8.1: A tree of 15 sites.

So, **when node 3 fails**, quorums can be formed:

$\{1,6,12,7,14\}$,
 $\{1,6,12,7,15\}$,
 $\{1,6,13,7,14\}$,
 $\{1,6,13,7,15\}$,
 $\{1,2,4,8\}$,
 $\{1,2,4,9\}$,
 $\{1,2,5,10\}$,
 $\{1,2,5,11\}$.

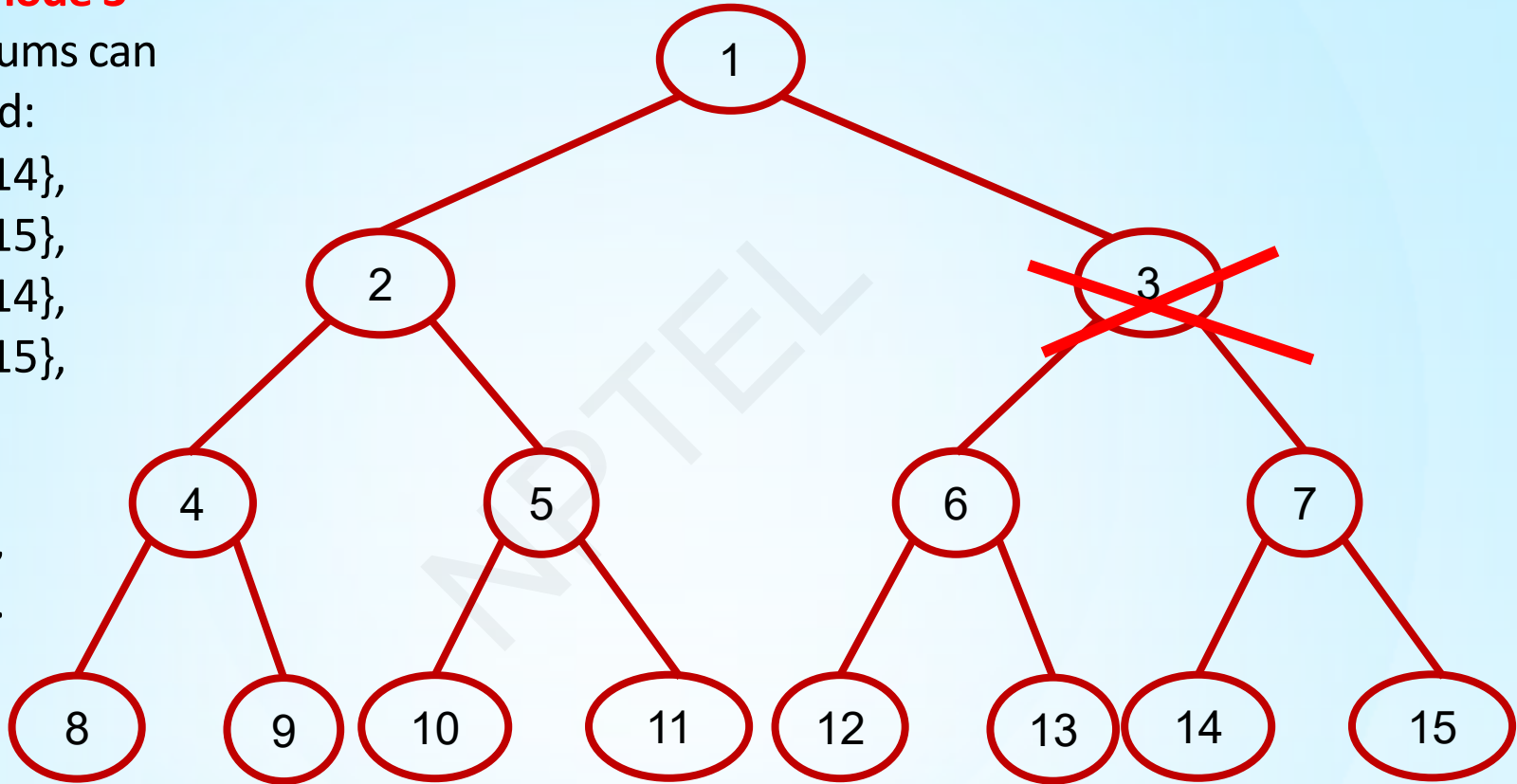


Figure 8.1: A tree of 15 sites.

Property of 'Graceful degradation'

- Since the number of nodes from root to leaf in an '**n**' node complete tree is **$\log n$** , the best case for quorum formation, i.e, **the least number of nodes needed for a quorum is $\log n$** .
- When the number of node failures is greater than or equal to **$\log n$** , the algorithm may not be able to form tree-structured quorum.
- So, as long as the number of site failures is less than **$\log n$** , the tree quorum algorithm guarantees the formation of a quorum and it exhibits the property of '**graceful degradation**'.

Mutual Exclusion Algorithm

A site s enters the critical section (CS) as follows:

- Site s sends a '**Request**' message to all other sites in the structured quorum it belongs to.
- Each site in the quorum stores incoming requests in a **request queue**, ordered by their timestamps.
- A site sends a '**Reply**' message, indicating its consent to enter CS, only to the request at the head of its **request queue**, having the lowest timestamp.
- If the site s gets a '**Reply**' message from all sites in the structured quorum it belongs to, it enters the CS.
- After exiting the CS, s sends a '**Relinquish**' message to all sites in the structured quorum. On the receipt of the '**Relinquish**' message, each site removes s 's request from the head of its **request queue**.

Mutual Exclusion Algorithm

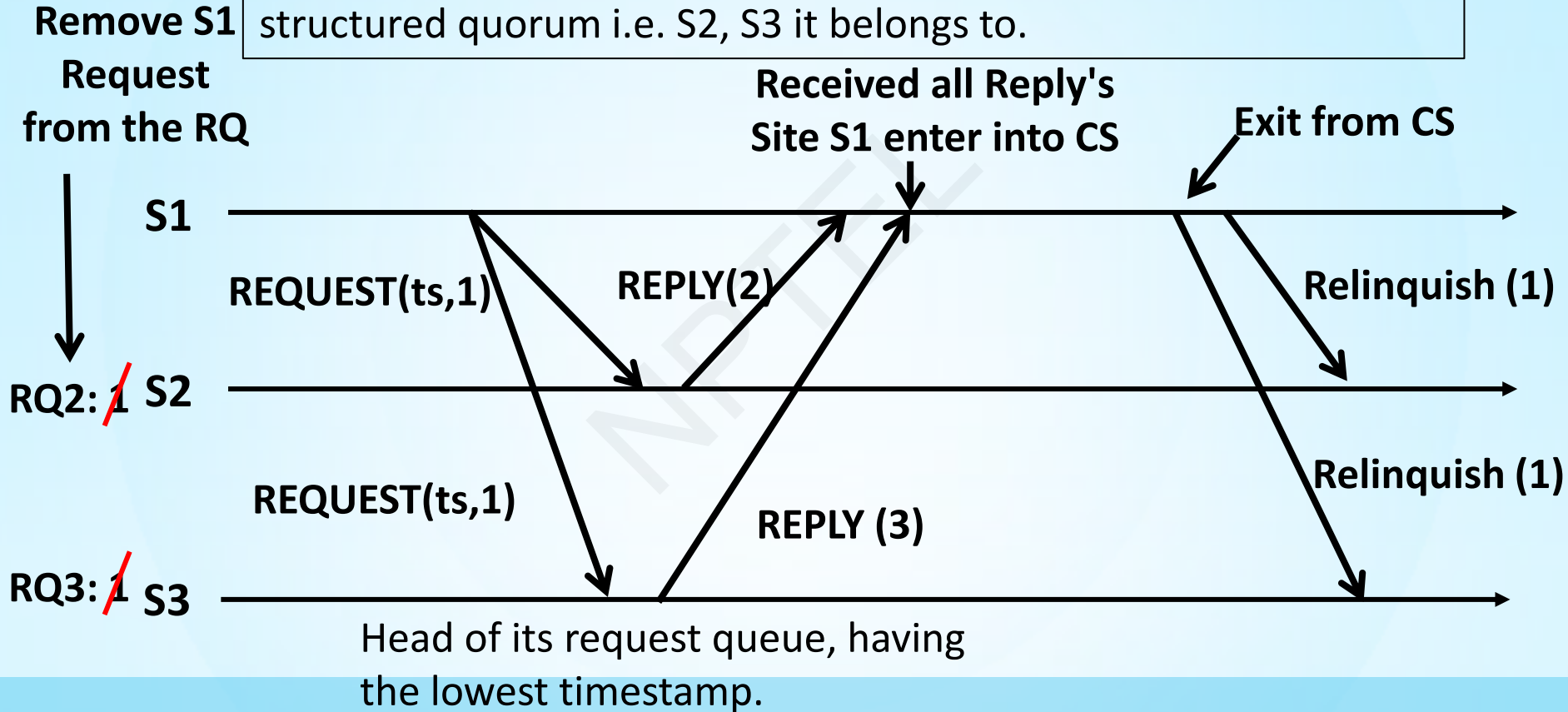
- If a new request arrives with a timestamp smaller than the request at the head of the queue, an **'Inquire'** message is sent to the process whose request is at the head of the queue and waits for a **'Yield'** or **'Relinquish'** message.
- When a site s receives an **'Inquire'** message, it acts as follows: If s has acquired all of its necessary replies to access the CS, then it simply ignores the **'Inquire'** message and proceeds normally and sends a **'Relinquish'** message after exiting the CS.

If s has not yet collected enough replies from its quorum, then it sends a **'Yield'** message to the inquiring site.

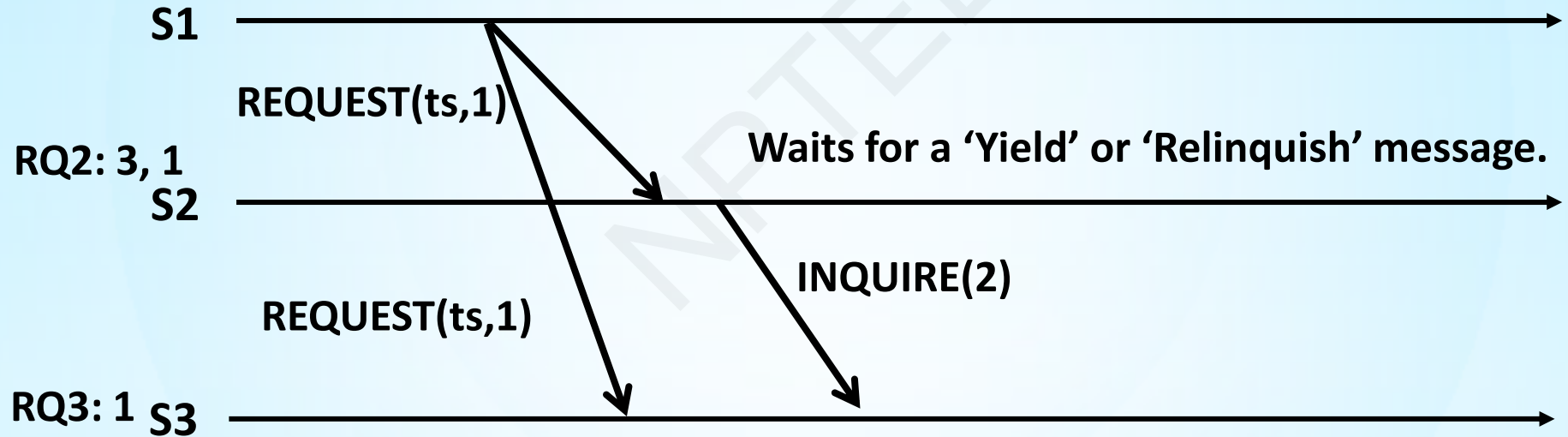
- When a site gets the **'Yield'** message, it puts the pending request (on behalf of which the **'Inquire'** message was sent) at the head of the queue and sends a **'Reply'** message to the requestor.

Example:

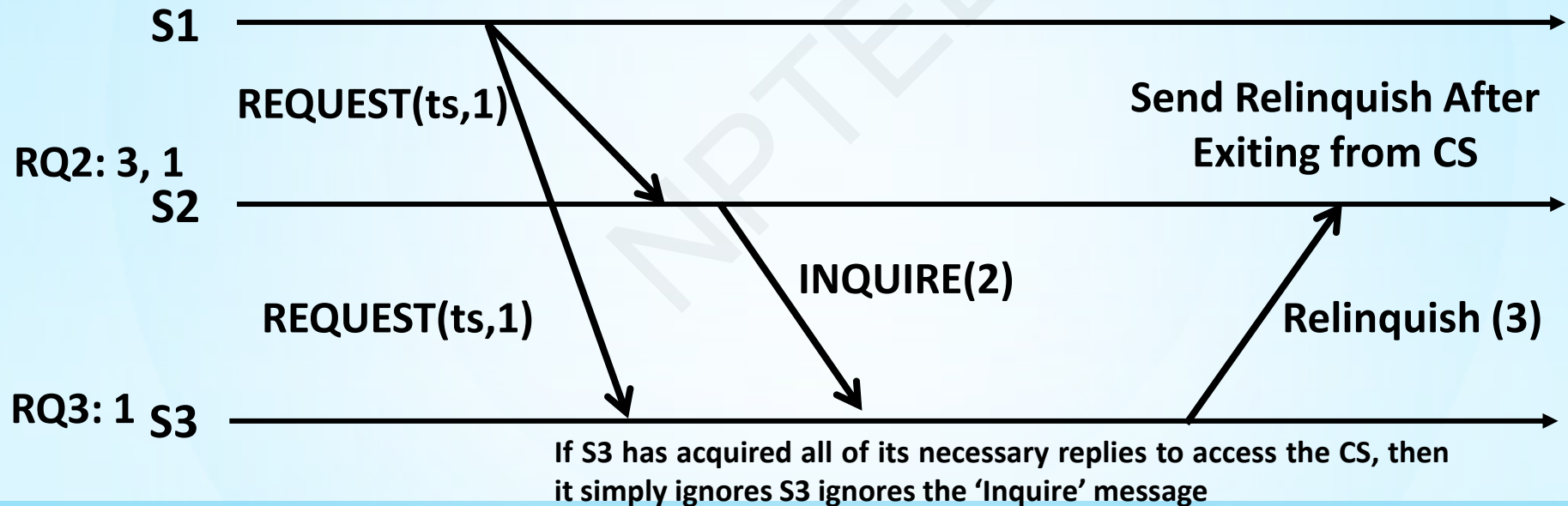
Site S1 sends a '**Request**' message to all other sites in the structured quorum i.e. S2, S3 it belongs to.



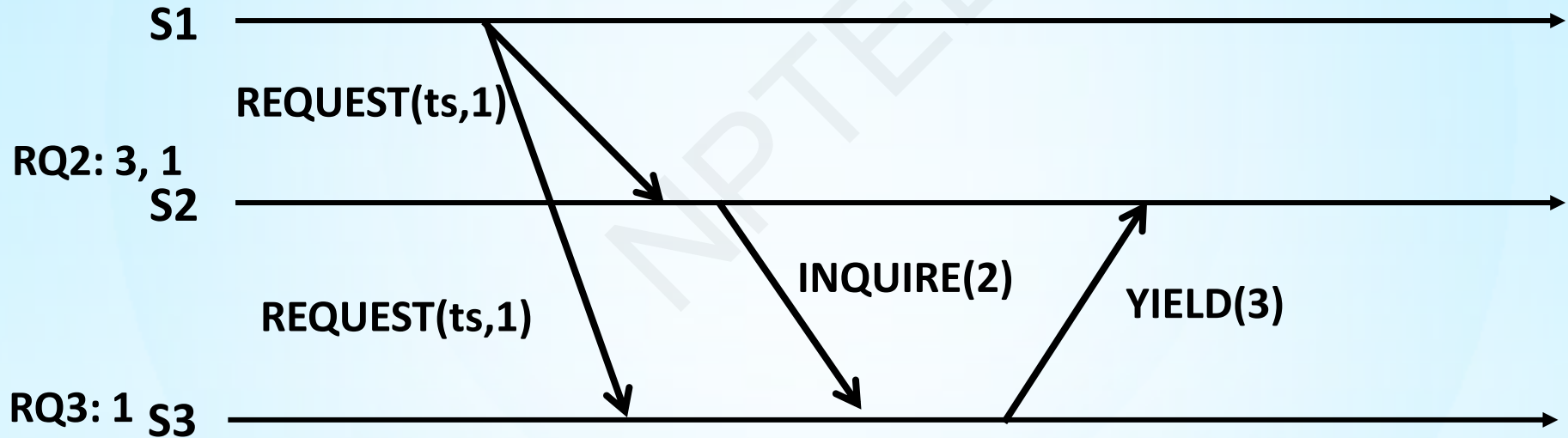
If a new request arrives with a timestamp smaller than the request at the head of the queue, an **'Inquire'** message is sent to the process whose request is at the head of the queue and waits for a **'Yield'** or **'Relinquish'** message.



When a site S3 receives an **'Inquire'** message, it acts as follows: If S3 has acquired all of its necessary replies to access the CS, then it simply ignores the **'Inquire'** message and proceeds normally and sends a **'Relinquish'** message after exiting the CS.

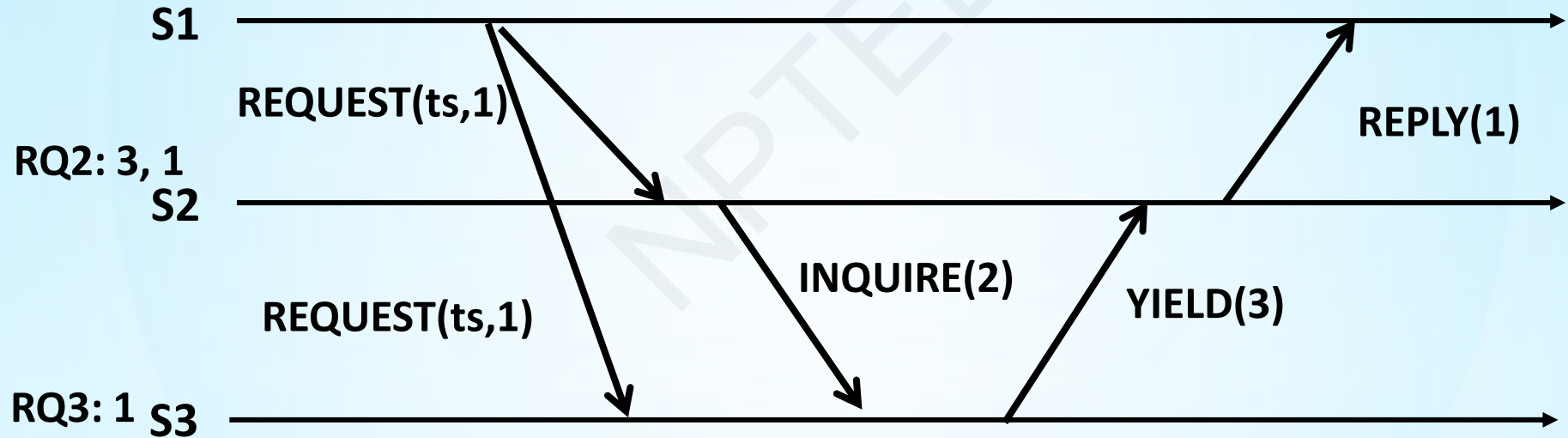


If S3 has not yet collected enough replies from its quorum, then it sends a **'Yield'** message to the inquiring site.



If S3 has not yet collected
enough replies from its quorum

When a site gets the '**Yield**' message, it puts the pending request (on behalf of which the '**Inquire**' message was sent) at the head of the queue and sends a '**Reply**' message to the requestor.



If S3 has not yet collected enough replies from its quorum

Correctness proof

- **Mutual exclusion** is **guaranteed** because the set of quorums **satisfy the Intersection property**.
- Consider a coterie C which consists of quorums $\{1,2,3\}$, $\{2,4,5\}$ and $\{4,1,6\}$.
- Suppose nodes 3, 5 and 6 want to enter CS, and they send requests to sites (1, 2), (2, 4) and (1, 4), respectively.
- Suppose site 3's request arrives at site 2 before site 5's request. In this case, site 2 will grant permission to site 3's request and reject site 5's request.
- Similarly, suppose site 3's request arrives at site 1 before site 6's request. So site 1 will grant permission to site 3's request and reject site 6's request.
- Since sites 5 and 6 did not get consent from all sites in their quorums, they do not enter the CS.
- Since site 3 alone gets consent from all sites in its quorum, it enters the CS and mutual exclusion is achieved.

Conclusion

- In this lecture, we have discussed about quorum based approaches. i.e. **Maekawa's Algorithm and Agarwal-El Abbadi Quorum-Based Algorithm**
- There exists a variety of quorums and a variety of ways to construct quorums. For example, **Maekawa used the theory of projective planes** to develop quorums of size $\sqrt{N}$ and Agarwal-El Abbadi quorum-based algorithm uses '**tree-structured quorums**'.
- In upcoming lecture, we will discuss about '**Token based approaches**' i.e. Suzuki-Kasami's Broadcast Algorithm and Raymond's Tree-Based Algorithm .